



QUEZON CITY UNIVERSITY



**SMARTSTOCK: An Inventory Management System
with QR-Code Integration**

A Capstone Project Documentation Presented to

The College of Computer Studies

QUEZON CITY UNIVERSITY

In partial fulfillment

of the requirements for the degree of

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

By

ALCANTARA, GEORGE HAROLD A.

ANGELES, RON PAULO G.

CALANGIAN, MATTHEW DANE D.

JARIEL, ALDRIN MIGUEL A.

PEREZ, JOHN CRISTOPHER A.

December 2025



APPROVAL SHEET

In partial fulfillment of the requirements for the degree **BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY**, this Capstone project entitled “**SMARTSTOCK: An Inventory Management System with QR-Code Integration**”, has been prepared and submitted by Alcantara, Harold George A., Angeles, Ron Paulo G., Calangian, Matthew Dane D., Jariel, Aldrin Miguel A., Perez, John Christopher A., who are hereby recommended for project presentation.

Mr. Joel M. Olayon

Capstone Project Mentor

Approved by the Committee for Project Presentation with a notation of

_____ on December 1, 2025.

Chairperson, Project Presentation Committee

Panel Member

Panel Member

Accepted and approved in partial fulfillment of the requirements for the degree

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY.

ISAGANI M. TANO, PHD-ELM, DIT

Dean of College of Computer Studies

Date: _____



DEDICATION

(can be tentative and to be completed in Capstone 2)



ACKNOWLEDGMENT

First and foremost, we thank **Almighty God** for granting us wisdom, perseverance, and strength throughout the duration of this study. Without His guidance and blessings, this project would not have been possible.

The researchers would like to express their gratitude to everyone who has contributed to the development of this study and system. Without the tremendous assistance of the researchers' valued professors, this paper and the system behind it would not have been possible.

The proponents would like to express their gratitude to Joel M. Olayon, a Capstone Project and Research Mentor, for his knowledge in providing suggestions, criticisms, and feedback, which has served as a source of inspiration and kept the researchers' work on track throughout the long process of developing the system and the gathering of foreign and local related literature for the final draft of this paper.

Lastly, the researchers would like to thank the administration of Quezon City University for allowing the students to contribute to the existing body of knowledge and assist future researchers in their research.



EXECUTIVE SUMMARY

Title : **SMARTSTOCK: An Inventory Management System
with QR-Code Integration**

Proponents : **Alcantara, George Harold A.**
Angeles, Ron Paulo G.
Calangian, Matthew Dane D.
Jariel, Aldrin Miguel A.
Perez, John Christopher A.

Mentor : **Joel M. Olayon**

Degree : **Bachelor of Science in Information Technology**

Date Completed : **December, 2025**

Keywords : **Inventory Management System, Java Swing, QR-Code Integration,
Role-Based Access Control (RBAC), Bulk Operations, Sales
Tracking, Multi-Store Support.**

(-----PROJECT BRIEF-----)



SUMMARY OF CHAPTER 1 TO 5

The study focused on developing **SmartStock**, a user-friendly inventory management system with QR-code integration, to address critical issues in traditional retail methods, such as manual tracking errors, slow transaction processing, and the lack of real-time stock visibility. It reviewed existing local and foreign systems to identify gaps in current solutions and adopted a developmental research approach using the **System Development Life Cycle (SDLC)** to strictly plan, design, and test the software. The project highlighted the inefficiencies of paper-based logbooks and proposed a secured digital solution featuring Role-Based Access Control and automated reporting. The system's three-tier architecture, normalized database, and intuitive Java Swing interface were designed to ensure data integrity, smooth operational workflows, and accurate financial tracking. The study concluded that the system is technically and economically feasible for implementation in SMEs and provided recommendations for future enhancements, such as cloud integration and mobile application development, to further support business growth.



EXECUTIVE SUMMARY

This study, titled SMARTSTOCK: An Inventory Management with QR-Code Integration, a Java Swing-based inventory and sales management system designed to solve problems like manual tracking errors and inefficient stock monitoring. The primary objective is to automate business processes using **QR-Code Integration**. The system aims to provide real-time inventory tracking, automatic profit calculation, and a secure environment for managing sales and product.

To ensure Data security, the system utilizes **Role-Based Access Control** that divides users into “Admin” and “Employee” roles. Admins have full control to manage store settings, add employees, and handle inventory, while employees are limited to processing sales and viewing basic reports. This structure prevents unauthorized changes, as Employees cannot delete products or adjust stock levels manually.

The system speeds up operations through **Bulk Operations**, allowing admins to select multiple products at once to delete them or update their stock levels simultaneously. The System features a smart 3-tier pricing system (Retail Price, Product Markup, and Default Markup) to automate pricing decisions. Lastly, the QR code integration is role-specific; Admins can generate and manage codes, while Employees can only use the scanner to sell products quickly.



TABLE OF CONTENTS

Title Page	i
Approval Sheet	ii
Dedication	iii
Acknowledgment	iv
Executive Summary	v
Table of Contents	vi
List of Appendices	vii

CHAPTER 1: CAPSTONE PROJECT BACKGROUND

Introduction	1
Project Context and Its Background	3
Objectives of the Study	4
Main Objectives	4
Specific Objectives	4
Significance of the Study	5
Scope and Limitations/Delimitations	7
Definition of Terms	8

CHAPTER 2: REVIEW OF RELATED LITERATURE AND SYSTEMS

Review of Related Literature	11
Local Literature	11
Foreign Literature	12
Review of Related Systems	14
Local Systems	14



Foreign Systems	15
Synthesis	16
CHAPTER 3: DESIGN AND METHODOLOGY	
Research Design	18
System Development Life Cycle (SLDC)	18
Figure 1: SLDC Diagram	19
Phase 1: Planning	20
Figure 2: Gantt Chart	20
Phase 2: Analysis	21
Phase 3: Design	21
Figure 3: System Architecture	22
Figure 4: Context Diagram	22
Figure 5: Use Case Diagram	23
Figure 6: Entity Relationship Diagram ...	24
Figure 7: System Flow Chart	25
Phase 4: Development	26
Phase 5: Testing	26
Phase 6: Implementation	27
Data Gathering Techniques	27
CHAPTER 4: SYSTEM ANALYSIS	
Current System Description	28
Proposed System Overview	29
Feasibility Study	30
Technical Feasibility	30



Operational Feasibility 30

Economic Feasibility 31

CHAPTER 5: SUMMARY, CONCLUSION, AND SUMMARY

Summary of Findings 32

Conclusion 32

Recommendations 33

References35



LIST OF APPENDICES

Appendix	Appendix Title	Page No.
A	Evaluation Form	39
B	User Interface	40
C	Documentation	44
D	Curriculum Vitae	45
E	Source Code	50



LIST OF FIGURES

Figure No.	Appendix Title	Page No.
1	SLDC Diagram	19
2	Gantt Chart	20
3	System Architecture	22
4	Context Diagram	22
5	Use Case Diagram	23
6	Entity Relationship Diagram	24
7	System Flow Chart	25



CHAPTER I

CAPSTONE PROJECT BACKGROUND

Introduction

Inventory management plays a crucial role in ensuring operational efficiency, preventing stock discrepancies, and supporting accurate organizational decision-making. In many small and medium-scale enterprises, traditional practices such as manual logging, spreadsheet-based tracking, and handwritten documentation continue to dominate, resulting in delays, human error, and limited real-time visibility of stock levels (Santos & Reyes, 2021). As business processes become more complex, these outdated methods can no longer keep pace with the growing need for speed, accuracy, and automation in inventory handling.

In recent years, digital transformation has accelerated the adoption of automated systems, particularly those incorporating barcode and Quick Response (QR) code technologies. QR codes have gained prominence due to their ability to store larger amounts of data, be scanned quickly using mobile devices, and integrate efficiently with modern database systems (Tan & Lee, 2020). These advantages make QR-code-based inventory management systems a practical solution for organizations seeking low-cost yet reliable automation tools.

However, despite the proven benefits of QR-code integration, many local businesses still experience challenges in implementing technology-driven inventory systems. Common issues include the absence of user-friendly interfaces, insufficient automation of stock monitoring, and the lack of real-time updates for incoming and outgoing items



(Dela Cruz, 2022). This gap highlights the need for an accessible, accurate, and streamlined solution that can assist businesses in managing inventory more efficiently.

The development of SMARTSTOCK: An Inventory Management with QR-Code Integration addresses these challenges by providing a system capable of automating stock tracking, generating QR codes for items, and enabling fast scanning for inventory transactions. By leveraging QR technology, the system aims to reduce human error, improve data accuracy, and provide timely insights into stock availability. This study aligns with existing literature emphasizing the importance of automated inventory systems in improving productivity and supporting data-driven business operations (Garcia & Mendoza, 2020).

Through the integration of modern features and user-centered design, SMARTSTOCK seeks to contribute to the ongoing shift toward digital inventory solutions and support organizations in optimizing stock management processes.



Project Context and Its Background

Effective inventory management is essential for business efficiency and productivity. It ensures that products are monitored, tracked, and replenished accurately to prevent losses and money mistakes. However, many small and medium-sized enterprises (SMEs) in the Philippines, as well as warehouses and schools, still rely on manual processes. These include handwritten logs, traditional stock cards, and basic spreadsheets (Santos & Reyes, 2021). These traditional methods often lead to human errors, mismatched stock levels, and delays. As businesses grow, these manual limitations cause bigger problems like overstocking, running out of items, and difficulty in retrieving records.

Digital technology has transformed how organizations manage inventory. Barcode and Quick Response (QR) code technologies are now widely used because they are low-cost and easy to use (Tan & Lee, 2020). QR codes are especially useful because they scan faster and store more data than old barcodes. Research shows that automated systems using QR codes improve accuracy, reduce workload, and help businesses make better decisions through fast data access (Garcia & Mendoza, 2020).

Despite these benefits, many local businesses still struggle to use automated tools. Common barriers include the high cost of systems, a lack of technical knowledge among employees, and the absence of user-friendly software made for small businesses (Dela Cruz, 2022). This creates a clear need for an affordable and accessible solution to solve the problems of manual inventory management.

To address these challenges, this project proposes SMARTSTOCK: An Inventory Management with QR-Code Integration. This digital platform automates stock monitoring and item tracking. By generating and scanning QR codes, the system ensures



fast item identification, accurate recording of stock-in and stock-out processes, and real-time updates on inventory status. SMARTSTOCK aims to minimize manual input, reduce human error, and help organizations operate more efficiently and sustainably.

Objectives of the Study

Main Objectives

To develop a user-friendly inventory management system with QR-code integration that simplifies stock monitoring, ensures accurate records, and enhances the operational efficiency of small and medium-sized enterprises.

Specific Objectives

1. Develop an automated inventory system that records and monitors stock-in and stock-out transactions in real time.
2. Integrate QR-code technology to streamline item identification, improve data accuracy, and minimize manual encoding errors.
3. Create a user-friendly interface that allows staff to easily add, update, search, and retrieve inventory information.
4. Generate automated inventory reports that provide accurate and updated information for decision-making.
5. Evaluate the system's overall performance, usability, and effectiveness through user testing and feedback.
6. Assess the technical, operational, and economic feasibility of implementing the system in a small to medium-scale organizational setting.



Significance of the Study

The development of **SMARTSTOCK: An Inventory Management with QR-Code Integration** holds significant value for various stakeholders who rely on accurate and efficient inventory processes. This study contributes both practical and academic benefits by addressing persistent challenges in manual inventory handling and promoting the adoption of digital solutions.

1. For Business Owners and Managers

This study provides business owners and managers with a system that enhances operational efficiency by automating stock monitoring and reducing the likelihood of human errors. With features such as QR-code scanning, real-time updates, and automated recording, SMARTSTOCK supports informed decision-making related to procurement, restocking, and resource allocation. Consequently, it helps businesses minimize losses associated with stock discrepancies and improve overall workflow management.

2. For Inventory Staff and Employees

Inventory personnel stand to benefit from a streamlined process that reduces the time and effort required for item tracking and documentation. The system simplifies routine tasks such as checking stock levels, updating records, and verifying item information. By minimizing manual input, the system not only increases accuracy but also lessens the workload and stress experienced by staff, thereby improving workplace efficiency and productivity.

**3. For Customers**

Indirectly, customers benefit from the system's contribution to more stable and reliable business operations. Accurate inventory tracking ensures timely availability of products, fewer stockouts, and improved customer satisfaction. Businesses adopting SMARTSTOCK are better positioned to maintain consistent service quality and meet customer demands efficiently.

4. For Future Researchers

The study provides a foundation for future academic work in inventory automation, QR-code integration, and digital business solutions. Researchers may use this project as a reference for developing enhanced systems, exploring emerging technologies such as RFID or IoT-based inventory monitoring, or conducting comparative studies on digital inventory management tools.

5. For the Academic

SMARTSTOCK serves as a concrete example of applying information technology to solve real-world organizational problems. It supports the academic community by showcasing the importance of integrating technological innovation with practical business needs. The system can be used as a learning material for IT students studying system development, database management, and process automation.

Overall, this study is significant as it promotes digital transformation among businesses and demonstrates the potential of QR-code technology to enhance inventory processes. By providing an accessible and efficient tool, SMARTSTOCK contributes to



both industry improvement and the advancement of knowledge in information systems development.

Scope and Limitations/Delimitations of the Project

Scope:

The system will cover the following:

1. The system will provide a complete inventory management solution specifically designed for small and medium-sized enterprises (SMEs).
2. The system will enable users to generate unique QR codes for products and scan them using a webcam for fast stock-in and stock-out (Selling) transactions.
3. The system includes a dashboard feature designed to provide users with a real-time view of sales, stock levels, and best-selling items.
4. The system can calculate accurate profits by tracking the historical cost of items and recording sales transactions.
5. The system will provide an export feature, allowing users to save inventory and sales reports as Excel (CSV) files for external record-keeping.
6. The system will support secure user management with distinct roles for Admins (full access) and Employees (limited access).
7. The system can categorize products and units to keep the inventory organized and easy to search.

Limitations | Delimitations:

The system is limited to the following:



1. The system will exclusively operate as a local desktop application. It will not use cloud storage or the internet, meaning data cannot be accessed from outside the specific computer it is installed on.
2. The system does not support managing multiple store branches simultaneously; it is designed for a single store location only.
3. The system will only work if a functional webcam is connected to the computer. Without a webcam, the QR scanning features cannot be used.
4. The system will not handle online payments, payroll, or advanced financial accounting such as tax calculations.
5. The system requires manual data entry for adding new products for the first time before a QR code can be generated.
6. The system allows export to Excel but does not support importing data from external Excel files back into the system.

Definition of Terms

In this study this section explains the important words used in the study so that one can easily understand how each term is used in the Smart Stock and Inventory Management System. The meanings are kept simple to avoid confusion and to make the concepts clear.

Inventory

In this study, inventory refers to the complete list of products or items stored, sold, or managed by the business, including their quantities and status.

Stock Monitoring



Stock monitoring refers to the continuous tracking of item levels to ensure that products are updated whenever stocks enter or leave the business.

Stock In / Stock Out

Stock In refers to adding new quantities of items into the inventory, while Stock Out refers to deducting quantities when items are sold, used, or transferred.

Smart Stock System

This refers to the computerized or digital system developed in the study that automates recording, updating, and managing inventory information.

User Access

User access refers to the level of permission given to different users of the system, such as Admin and Staff, which controls what information they can view or update.

Dashboard

In this study, a dashboard is the real-time visual display that shows the current status of inventory, such as total items, low-stock products, and recent transactions.

QR Code

These refer to the scannable codes used to quickly identify and record product information in the system.

Inventory Report



An inventory report is the generated summary of stock-related information, showing item quantities, stock history, and overall inventory status.

Database

The database refers to the storage system where all inventory records, product information, and user data are saved and organized.

SMEs (Small and Medium-sized Enterprises)

Refers to the small businesses (like retail stores) that are the main beneficiaries of this study, which currently rely on manual inventory methods.

Java

Refers to the object-oriented programming language used to develop the SMARTSTOCK desktop application.



CHAPTER II

REVIEW OF RELATED LITERATURE AND SYSTEMS

Review of Related Literature

Local

McCloud et al. (2023) emphasized the critical role of systematic data collection in facilitating responsive feedback and evidence-based decision-making. Their research highlights that consistent and organized data gathering is essential for organizations to generate reliable insights and make informed operational adjustments. This supports the fundamental premise that automated systems, by ensuring data accuracy and availability, are vital components for effective organizational management and strategic planning.

Masih (2022) investigated the feasibility of utilizing QR codes for the registration and evaluation of training programs, specifically analyzing learners' perceptions. The study found that QR code implementation significantly streamlined the registration process and increased response rates for evaluations. The findings highlighted the technology's ability to enhance user engagement and efficiency, suggesting its broader potential for improving administrative workflows and data collection in educational and organizational settings.

Llabres (2023) developed an *Auto Parts Inventory Management System* to address the specific inventory challenges faced by automotive businesses in the Philippines. The study focused on transitioning from manual monitoring to a digital solution, aiming to improve the accuracy of tracking vehicle components, reduce discrepancies in stock records, and streamline stock replenishment processes for local retailers.



Pabillaran et al. (2023) assessed the efficiency, usability, and acceptance of QR-code-based digital systems implemented in a local government setting. The study revealed that QR systems significantly reduced processing time and errors associated with manual recording. Users reported high satisfaction due to the system's convenience, accuracy, and ease of use. Recommendations focused on improving user interface design and expanding system deployment to additional government units.

Hortizuela (2019) implemented a QR-powered monitoring system for school facilities and equipment, enabling accurate tracking of maintenance schedules, condition updates, and asset locations. The system helped administrators reduce delays in equipment servicing and improve preventive maintenance. User feedback highlighted enhanced efficiency and reduced clerical work, with suggestions to adopt cloud-based storage for multi-campus accessibility.

Foreign

Noguerra (2023) designed and evaluated an innovative mobile solution utilizing QR codes to streamline inventory monitoring and address manual tracking inefficiencies. The system featured modules for stock-in and stock-out management, item monitoring, and real-time inventory reporting, all secured through user authentication. Evaluation results yielded an overall rating of 4.5 out of 5, receiving perfect scores for efficiency (5/5) and security (5/5). The study concluded that the system effectively minimized stockouts, optimized inventory processes, and facilitated informed decision-making.

Moh Fajar et al. (2025) explored the optimization of inventory management in a regional revenue office by integrating QR codes with a sequential search algorithm. Their study demonstrated that combining these technologies significantly improved the speed



and accuracy of locating assets within a large database. The research highlighted that while QR codes provide rapid physical identification, the underlying search algorithm is crucial for retrieving digital records efficiently in high-volume inventory systems.

Yaqin et al. (2024) implemented a QR-based inventory tracking system for a health sciences institution, focusing on consumable materials. The system automated stock alerts, improved accuracy of supply counts, and centralized all inventory data through a web platform. Evaluation showed significant improvements in transparency, reduced shortages, and streamlined updates. The study suggested integrating predictive analytics for future expansions.

Balaji et al. (2024) proposed a web-based inventory platform enhanced with QR-code scanning to improve traditional business stock management. The system enabled automated input, reduced documentation errors, and provided real-time visibility of stock levels. Findings revealed enhanced workflow efficiency and better traceability of items. Researchers recommended mobile optimization to increase system flexibility across devices.

Shirley et al. (2021b) developed the QR-Based Inventory Management System (QR-IMS), a solution specifically designed for tracking passenger luggage via a website interface. By utilizing QR codes for identification and monitoring, the system addresses common challenges in luggage handling, aiming to improve security and efficiency in transport sectors.



Review of Related Systems

Local Systems

HashMicro Inventory System (Philippines) is a cloud-based enterprise inventory solution widely used by Philippine businesses. It features barcode and QR-code integration, automated stock tracking, multi-warehouse management, and analytics dashboards. The system improves operational efficiency and reduces manual workload, making it suitable for medium to large organizations. Its relevance lies in demonstrating how automated inventory platforms can streamline stock movement across different departments (HashMicro, 2024).

QNE Inventory Management System is a Philippine-based solution tailored for local distributors and retailers. It provides item categorization, unit measurement control, barcode generation, and basic automation of inventory transactions. The system has been widely adopted due to its user-friendly design and integration with accounting modules. Its application highlights the importance of digitizing traditional stock card methods (QNE Software Philippines, 2024).

SiteGiant Philippines is an e-commerce and inventory management platform used by online sellers and SMEs. It supports inventory synchronization across local marketplaces such as Shopee and Lazada. Users reported improved sales tracking and reduced overselling due to real-time updates. This system illustrates the value of centralized tracking for businesses with multiple sales channels (SiteGiant, 2024).

StoreHub (Southeast Asia / Philippines) is a POS and inventory management system used by Philippine retailers and restaurants. It offers barcode scanning, ingredient-level



tracking, and automated stock deductions for sales. The system is known for its intuitive interface and mobile accessibility. Its relevance lies in its integration of sales and inventory functions (StoreHub, 2024).

Peddlr Inventory for SMEs is a mobile-first POS and inventory tool designed for small Philippine businesses. It provides simple inventory tracking, item monitoring, and QR-capable product listings. Its wide adoption among small retailers highlights the growing demand for affordable, digitally accessible inventory systems (Peddlr, 2024).

Foreign Systems

Zoho Inventory is a global cloud-based inventory solution offering barcode and QR-code support, order management, warehouse tracking, and automated reordering. Its scalability makes it popular among SMEs worldwide. Zoho demonstrates how automation improves accuracy in multi-location inventory (Zoho Corporation, n.d.).

inFlow Inventory is an international inventory control software featuring barcode scanning, stock monitoring, and order fulfillment tools. Users report faster processing times and improved stock visibility. It serves as a model for integrating desktop and cloud functionalities in one platform (inFlow Inventory, 2024).

Odoo Inventory is part of the Odoo ERP suite and supports advanced warehouse automation, barcode scanning, routing, and replenishment rules. Its modular nature shows how inventory systems can expand to serve complex operations such as manufacturing and distribution (Odoo, 2024).

Fishbowl Inventory is widely used in manufacturing and warehouse environments. It includes barcode/QR support, lot and serial tracking, order management, and inventory



reports. Its integration with accounting platforms like QuickBooks demonstrates the importance of linking inventory with finance (Fishbowl, 2024).

QuickBooks Commerce (formerly TradeGecko) offers automated stock updates, barcode support, order tracking, and reporting for global SMEs. It is known for real-time synchronization across multiple online stores. This system serves as a reference for businesses transitioning from manual to cloud-based automation (Intuit QuickBooks, 2024).

Synthesis

In this Chapter, the review of related literature and systems explains why automated inventory systems are important. The studies discussed here give useful ideas about inventory control. They show how using QR codes, cloud tracking, real-time monitoring, and mobile apps can help businesses switch from paper records to better digital ones.

Local literature in the Philippines, including McCloud et al. (2023) and Pabillaran et al. (2023), demonstrates that digital data collection and QR systems significantly enhance operational feedback and reduce manual errors. Pabillaran et al. (2023) specifically proved that such systems improve processing time and user satisfaction in local government settings. Additionally, Hortizuela (2019) confirms that QR codes streamline asset tracking and reduce clerical workload in educational facilities.

In foreign literature, research by Noguerra (2023), Yaqin et al. (2024), and Balaji et al. (2024) illustrates how connecting mobile phones and web platforms to a central system eliminates human errors. These studies show that data becomes clearer and more accurate when automated. Specifically, Yaqin et al. (2024) and Balaji et al. (2024)



highlight improvements in workflow efficiency and traceability across health and business sectors. Shirley et al. (2021b) further validates the versatility of QR technology in specialized tracking applications like passenger luggage.

The review of existing systems shows how these technologies work in the real world. Local apps like HashMicro and Peddlr prove that cloud-based tracking helps Philippine businesses grow and manage stock smoothly. Meanwhile, foreign platforms like Zoho Inventory and Odoo show advanced ways to manage warehouses. They also demonstrate why it is important to connect inventory data with sales and financial records to handle complex operations.

Overall, this chapter shows that it is important to keep researching and developing inventory systems to meet the changing needs of businesses. Improving the accuracy, speed, and reliability of stock management is essential. This analysis highlights how modern tools like QR codes and mobile apps make tracking easier while keeping data safe. By reviewing these studies and systems, the chapter confirms the need for new, smart solutions to help small businesses work more efficiently.

<end of page>



CHAPTER III

DESIGN AND METHODOLOGY

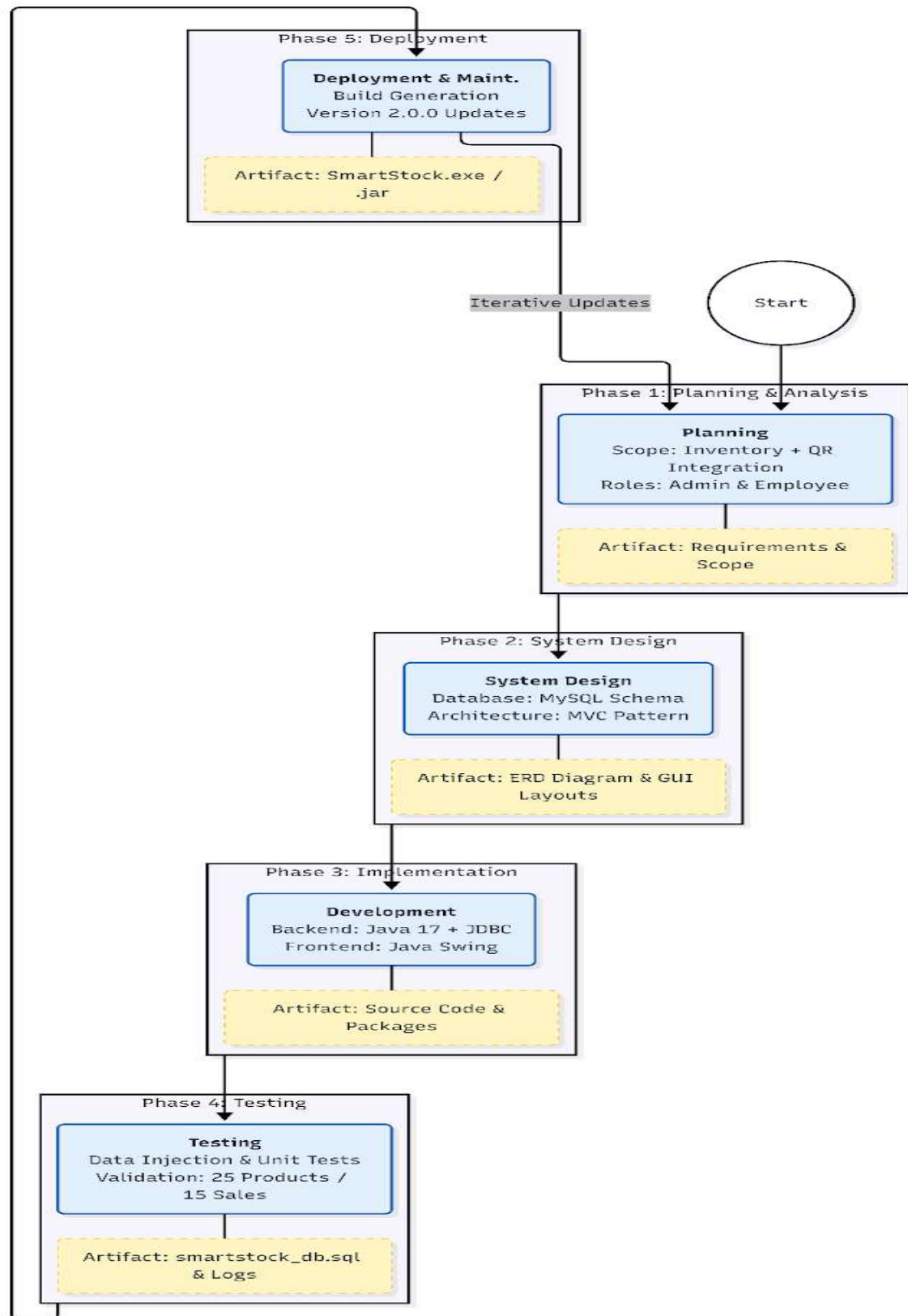
Research Design

This study employed a Developmental Research Design, specifically utilizing the **System Development Life Cycle (SDLC)** approach to design, develop, and evaluate the *SMARTSTOCK: An Inventory Management with QR-Code Integration*. The developmental design was chosen because the study's primary goal was to create a functional and efficient software solution that automates stock monitoring and sales recording processes while reducing manual errors and workload.

The SDLC framework guided the systematic development of the system through the following phases: **(1) Planning, (2) Analysis, (3) Design, (4) Development, (5) Testing, and (6) Implementation.**

System Development Life Cycle (SLDC) Approach

The researchers utilized the System Development Life Cycle (SDLC) approach as the methodological framework for the development of the SMARTSTOCK: An Inventory Management with QR-Code Integration. The SDLC is a structured process used in system development that ensures a project's efficiency, functionality, and quality through a systematic progression of phases. It provides a clear framework for planning, designing, building, testing, and deploying an information system that meets the users' requirements.

FIGURE 1: SLDC DIAGRAM



1. Planning Phase

In this initial stage, the researchers identified the major problems with manual inventory systems, such as inaccurate stock tracking, time delays in recording transactions, and difficulty in generating reports. The team established the project's main objective: to build a solution specifically for Small to Medium Enterprises (SMEs) using a Java-based system that utilizes QR codes to replace handwritten logs. This phase set the clear goal of creating an automated tool to prevent losses and money mistakes caused by traditional methods.

Figure 2: Gantt Chart



The Gantt chart illustrates the 12-week development schedule for the SmartStock Capstone project, structured around the System Development Life Cycle (SDLC). The project begins with Planning and Analysis (Weeks 1-2), focusing on defining



requirements and database structure. It progresses into Design and Development (Weeks 2-8), where the team builds the core Java application, integrates the MySQL database, and implements key features like the Dashboard, Sales Module, and QR Code scanning. Testing (Weeks 9-10) involves User Acceptance Testing (UAT) and bug fixing to ensure system stability. Finally, the Implementation phase (Weeks 10-12) covers the final system build, hardware setup, and defense rehearsals, concluding with the Capstone Final Defense in Week 12.

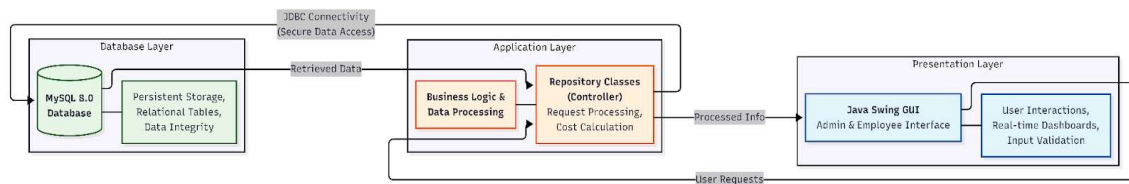
2. **Analysis Phase**

During the analysis phase, the team gathered detailed requirements through interviews and observations to understand exactly what features the system needed. We determined that the system requires a Webcam Scanner for effective and fast real-time scanning and established two distinct user roles—Admin (full access) and Employee (limited access)—to ensure data security. The analysis also confirmed the need for specific business tools, such as Bulk Operations for managing many products at once and CSV Export capabilities to allow users to view reports in Microsoft Excel.

3. **Design Phase**

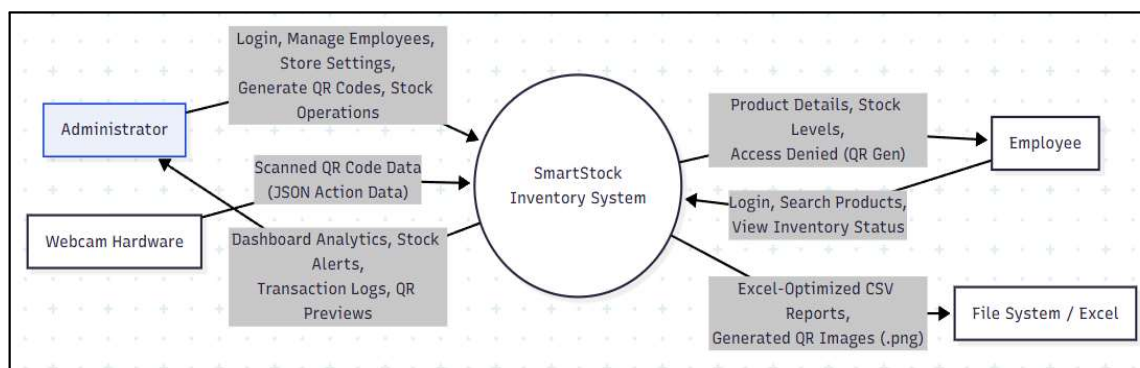
In this phase, the researchers designed the system's architecture, database structure, and user interface to serve as a blueprint for development. We designed the User Interface (UI) using Java Swing for a user-friendly desktop experience and structured the MySQL database with 8 core tables including users, stores, products, and sales. Data Flow Diagrams (DFDs) and Entity-Relationship Diagrams (ERDs) were created to visualize how data would move securely from the user input to the database storage.

System Architecture



The System Architecture of the SmartStock Inventory Management System is organized into a three-tier structure comprising the Presentation Layer, Application Layer, and Database Layer. The Presentation Layer utilizes Java Swing to provide an interactive GUI for Admins and Employees, handling all user input and visualization. The Application Layer manages business logic and data processing through Repository classes, acting as a bridge to calculate costs and process requests. Finally, the Database Layer employs MySQL 8.0 for persistent storage, ensuring data integrity and secure retrieval via JDBC connectivity.

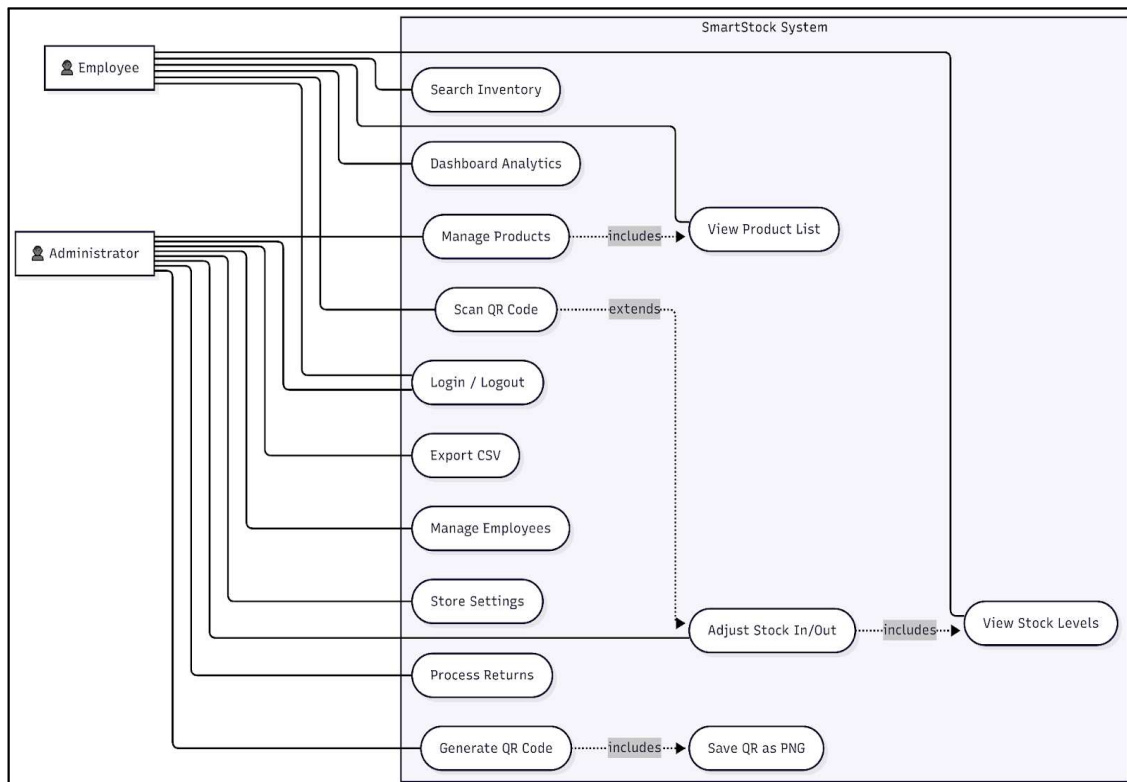
Context Diagram



The Context Diagram illustrates the high-level boundary of the SmartStock Inventory Management System, identifying the Administrator and Employee as the primary users who provide inputs such as login credentials and stock adjustments in exchange for real-time analytics and inventory status. It further highlights the system's

integration with external hardware and storage: the Webcam inputs scanned QR code data directly into the application, while the File System serves as the output destination for exported Excel-optimized CSV reports and generated QR code images.

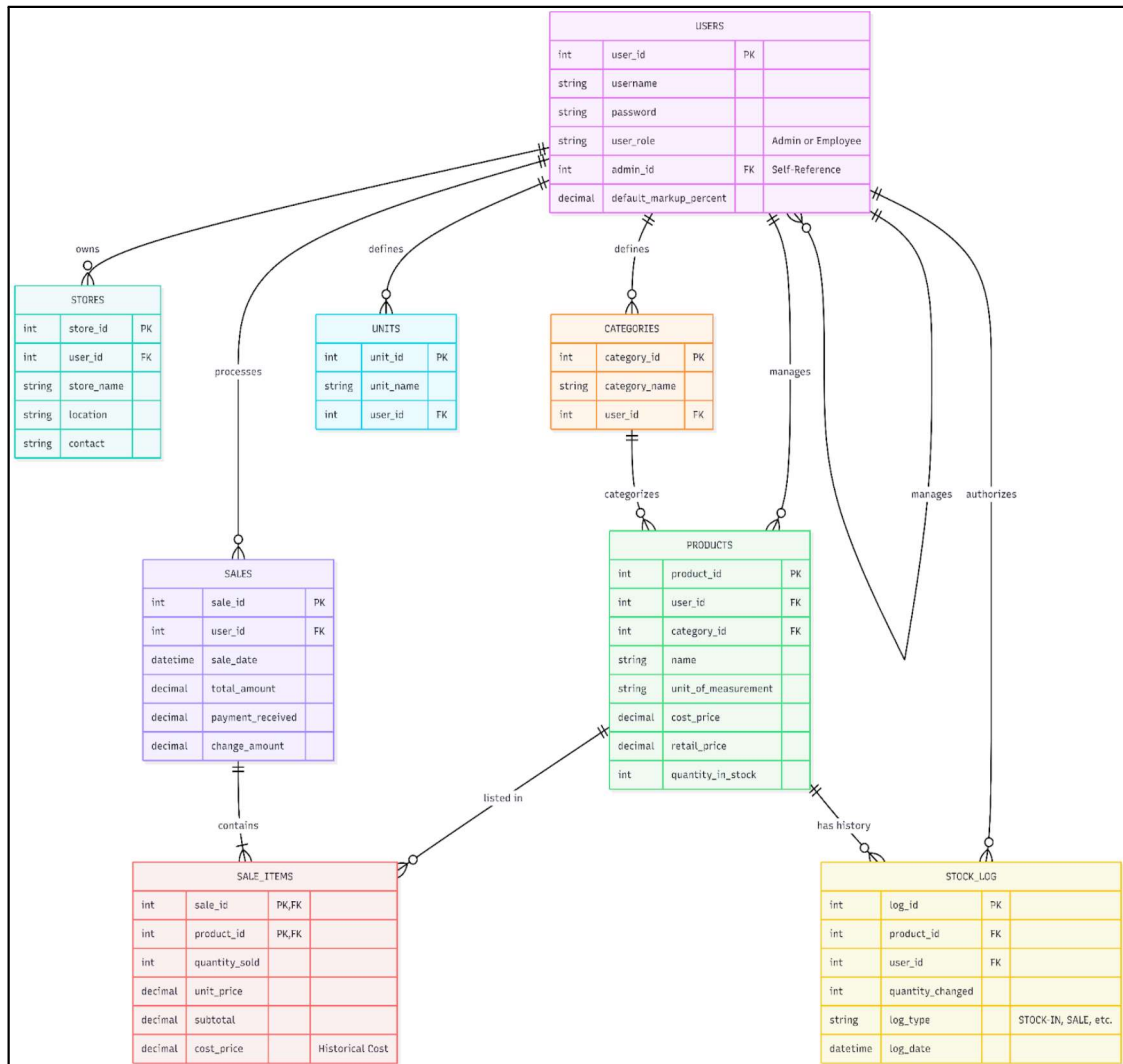
Use Case Diagram



The Use Case Diagram outlines the functional requirements of the SmartStock Inventory Management System, establishing clear security boundaries through Role-Based Access Control (RBAC). It differentiates between the Administrator, who possesses extensive privileges for critical operations such as store configuration, employee management, and manual stock adjustments, and the Employee, whose access is strictly limited to "view-only" functions like searching the product catalog and monitoring stock levels to preserve data integrity.

Database Design

ERD Diagram (Entity Relation Diagram)

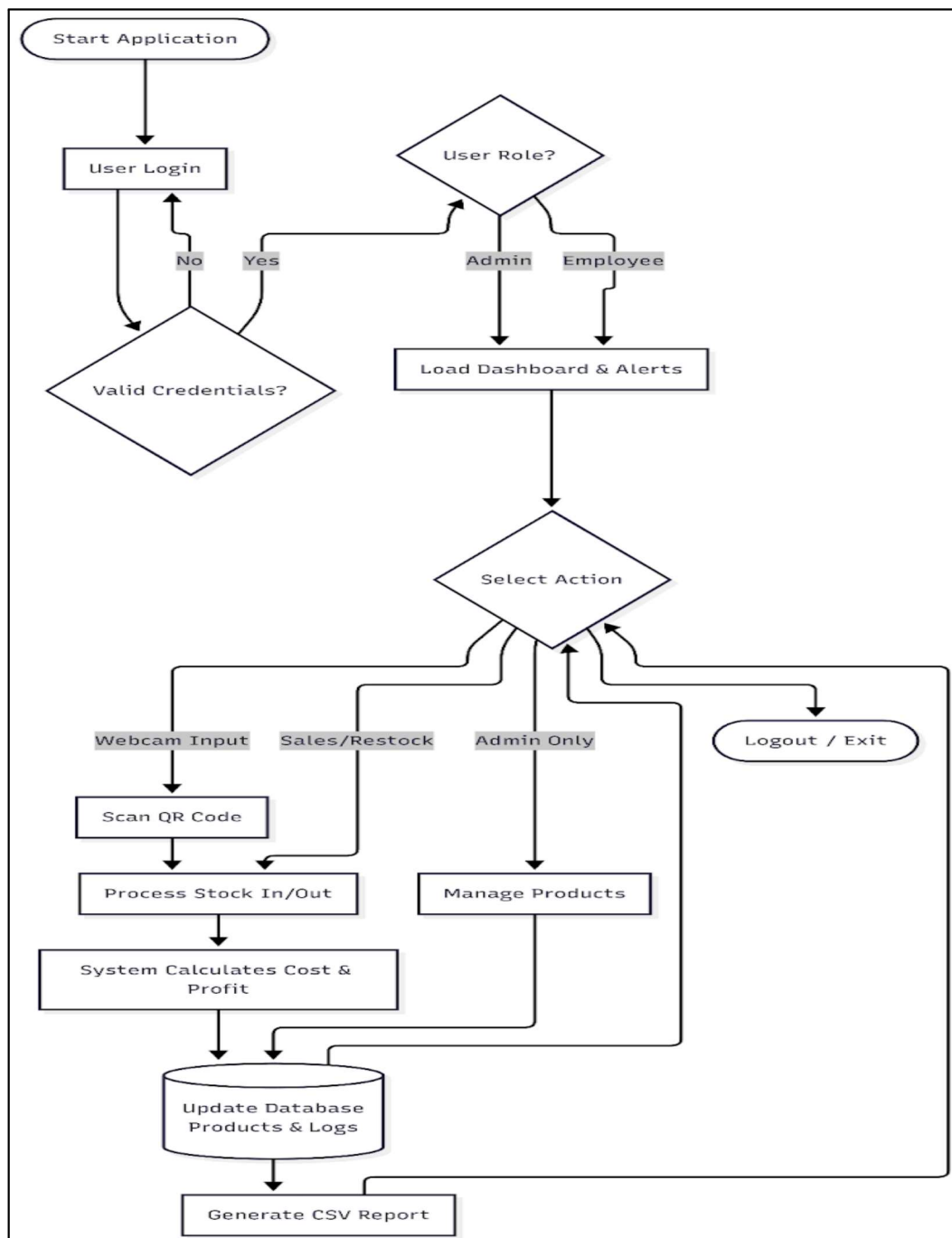


The Entity Relationship Diagram (ERD) illustrates the normalized database schema of the SmartStock System, centered on the users entity which utilizes a self-referencing relationship to implement Role-Based Access Control (RBAC) for Admins and Employees. The inventory architecture is built around the products entity, supporting dynamic classification via categories and flexible quantization through units, while financial integrity is secured by the sales and sale_items tables which preserve historical



pricing data. Additionally, the stock_log entity provides a comprehensive audit trail, recording every inventory movement (such as stock-ins, sales, and returns) to ensure complete traceability of assets.

Flow Chart





The System Flowchart shows how the SmartStock system works step-by-step. It starts with the login screen, where the system checks if the user is an Admin or an Employee. Then, it moves to important tasks like scanning QR codes to identify products and processing sales transactions. Finally, it saves all the data into the database and creates reports to download, ensuring that the inventory numbers are always accurate and up-to-date.

4. Development Phase

This stage involved the actual coding and configuration of the SmartStock system using the planned specifications. The team built the software using Java 21 as the main programming language and XAMPP (MySQL 8.0) for backend database management. To enable the system's core features, developers integrated specific libraries such as ZXing 3.5.0 for smart QR code generation and decoding, Webcam Capture 0.3.12 for real-time camera scanning, and JCalendar for date selection tools.

5. Testing Phase

During the Testing Phase, the researchers validated the SmartStock system's functionality and reliability through Unit, Integration, and User Acceptance Testing to ensure it met all operational requirements. Unit Testing verified individual features, such as the Smart Pricing logic for automatic retail calculations and strict Input Validation to prevent errors like negative stock values. Integration Testing confirmed the secure connection between the Java Swing GUI and the MySQL database, as well as the compatibility of the Webcam Capture and ZXing libraries for accurate real-time QR code scanning. Finally, User Acceptance Testing (UAT) simulated real-world retail scenarios, where the team successfully executed Stock-Out and Return transactions, performed Bulk



Operations, and verified that CSV Exports were correctly formatted for Microsoft Excel compatibility.

6. **Implementation Phase**

The final phase involved deploying the SmartStock system for actual operational use. The final software was packaged as a SmartStock.jar file, converting it into an executable file, and users were guided to install the necessary Java Development Kit (JDK 21) and XAMPP tools. This phase also included configuring the database connection via the config.properties file and training users on how to operate the system, such as using the webcam scanner and managing restricted Employee Accounts.

Data Gathering Techniques

Since this project focuses on system development, the researchers did not conduct surveys or interviews. Instead, the system was developed by analyzing standard inventory workflows and studying existing processes. The team observed common problems in manual stock tracking and sales recording, and used these observations as the basis for the system's design.

<end of page>



Chapter IV

SYSTEM ANALYSIS

Current System Description

The current inventory management process used by many small businesses is primarily manual and paper-based. It relies on handwritten logs, basic spreadsheets, and physical record books to track products. Inventory officers record details such as product name, quantity, supplier, and transaction dates by hand. This method often leads to inconsistent records and delays. Stock monitoring is done by physically counting items in storage and comparing them to the logbook. Because there is no automated tracking, business owners have limited visibility of their real-time inventory levels, making it hard to spot shortages or overstocking immediately.

Additionally, this manual process requires staff to repeatedly rewrite data for audits and monthly reports. These repetitive tasks increase the risk of human error, such as miscalculations and illegible handwriting. Retrieving information is also time-consuming, as staff must flip through pages of logs or search through multiple spreadsheet files to find specific item details.

In terms of security, the current system offers little protection. Since physical logbooks and basic spreadsheet files are not secured, anyone can change or remove entries without being traced. There are no automatic audit trails, making it difficult to verify transactions or hold personnel accountable for discrepancies. Overall, the existing system is slow, error-prone, and inefficient, highlighting the urgent need for a QR-code-enabled solution.



Proposed System Overview

The proposed system, SMARTSTOCK: An Inventory Management with QR-Code Integration, is designed to replace these manual processes with an automated and efficient solution. The system introduces a centralized digital platform that enables real-time monitoring and tracking of inventory through QR-code technology. Its goal is to streamline data entry, improve stock visibility, and minimize human error.

At the core of SMARTSTOCK is its QR-code tracking capability. The system generates a unique QR code for each item, containing essential details like product name, category, and quantity. Using a webcam-based scanner, users can quickly identify items, update stock counts, and process sales transactions. This feature significantly reduces the time needed for encoding and ensures data accuracy.

The system also includes a secured MySQL database that stores all records. Automated updates ensure that every sale or delivery is immediately reflected in the system. This gives business owners accurate, real-time data for better planning.

Moreover, SMARTSTOCK improves security through User Authentication. It distinguishes between "Admin" and "Employee" roles, ensuring that only authorized personnel can modify critical data. The system also maintains an audit trail of user activities to enhance accountability. Finally, the system includes automated report generation, allowing users to export inventory summaries and sales logs to Excel (CSV) for easier analysis and decision-making.

**Feasibility Study (Technical, Operational, Economic)****Technical Feasibility**

The proposed SMARTSTOCK system is technically feasible because it uses stable and widely available technologies. The system is built using Java 21 and MySQL (via XAMPP), which are standard, reliable, and open-source tools supported by most desktop computers. The hardware requirements are minimal, requiring only a standard desktop or laptop and a basic webcam for the scanning feature, eliminating the need for expensive specialized barcode scanners. Furthermore, the development team has the necessary skills to integrate the Webcam Capture and ZXing libraries, ensuring that the QR code generation and scanning features will work smoothly without requiring complex infrastructure.

Operational Feasibility

Operational feasibility evaluates if the system can be effectively used by the organization. SMARTSTOCK is designed with a user-friendly Java Swing interface that is simple and intuitive, allowing staff with minimal technical skills to operate it easily. The system directly addresses current operational problems—such as slow manual tracking and human error—by automating stock updates and sales recording. With features like Role-Based Access Control, the system fits well into standard business hierarchies, giving Admins full control while restricting Employees to sales tasks. Since the system simplifies daily tasks rather than making them harder, it is highly likely to be accepted and adopted by business owners and staff.



Economic Feasibility

From an economic perspective, **SMARTSTOCK** is a highly cost-effective solution. The development costs are very low because the system utilizes free, open-source software (Java, MySQL, XAMPP) and runs on existing computer hardware. The only potential cost is the acquisition of a webcam if one is not already available. In the long run, the system saves money by preventing inventory losses due to theft, misplacement, or overstocking. By providing accurate reports and real-time data, business owners can make better financial decisions, ensuring a high return on investment compared to the inefficiencies of the manual system.



Chapter V

Summary, Conclusion, & Recommendations

Summary of findings

This study focused on the development of SMARTSTOCK, an inventory management system designed to solve the problems of manual tracking in Filipino Small and Medium Enterprises (SMEs). The researchers utilized the System Development Life Cycle (SDLC) to plan, design, build, and test the software. The study found that traditional methods, such as handwritten logs and spreadsheets, are prone to human error and lack real-time visibility . Through the development process, the team successfully created a Java-based application integrated with QR-code technology and a MySQL database. The testing phase, which included Unit, Integration, and User Acceptance Testing, confirmed that the system effectively automates stock-in and stock-out operations, ensures data accuracy, and generates reliable sales reports. The results showed that the system functions correctly on standard desktop computers using a webcam for scanning, meeting the technical requirements established at the beginning of the study .

Conclusion

Based on the findings, the researchers conclude that SMARTSTOCK is a highly effective solution for modernizing the operations of small retail businesses. The integration of QR-code technology successfully eliminates the inefficiencies of manual data entry, significantly reducing processing time and human errors . The system's Role-Based Access Control (RBAC) improves security by restricting sensitive tasks to Admins, while the Real-Time Dashboard provides business owners with immediate



insights into their sales and stock levels, which was not possible with paper records. Furthermore, the system's ability to export data to Excel (CSV) ensures that businesses can maintain organized records for accounting purposes. Overall, the project achieved its main objective of developing a user-friendly, accurate, and cost-efficient inventory tool that helps SMEs operate more professionally and sustainably.

Recommendations

Based on the conclusions and the limitations of the current system, the researchers offer the following recommendations for future improvement and implementation:

1. For Business Owners and SMEs: It is recommended to adopt this automated system to minimize losses caused by theft or miscounting. Owners should ensure that the computer hardware, specifically the webcam, is of good quality to ensure fast scanning. Regular data backups (exporting CSVs) should be performed to prevent data loss since the system is currently offline .
2. For Future Researchers (Cloud Integration): The current system is limited to a local desktop environment. Future researchers are encouraged to develop a cloud-based version or a web application. This would allow business owners to monitor their inventory remotely from any location using the internet and enable data synchronization across multiple store branches.
3. For Future Researchers (Mobile Application): While the current system uses a webcam, future developers should consider creating a companion mobile app. This would allow staff to use their smartphones as portable scanners for conducting inventory audits on the shelves, increasing mobility and convenience .



4. For Future Developers (Advanced Analytics): To further enhance decision-making, future versions of the system could include predictive analytics. This feature would analyze historical sales data to predict future demand and automatically suggest which items need to be reordered before they run out.

5. For System Security: Future studies should implement stronger data security measures, such as automated cloud backups and data encryption. This would protect the business's financial records from being lost in case of computer failure or hardware damage.



References

- Balaji, T., Hari, V., Lathifunnisa, S., Ganesh, P., & Arupya, P. (2024). Optimizing web-based inventory management system using QR code technology. 2024 7th International Conference on Circuit Power and Computing Technologies (ICCPCT), 751–756.
<https://doi.org/10.1109/iccpct61902.2024.10673103>
- Dela Cruz, P. R. (2022). Challenges in implementing technology-driven inventory systems in local businesses.
- Fishbowl Inventory. (2024). Fishbowl inventory software.
<https://www.fishbowlinventory.com/>
- Garcia, R. J., & Mendoza, L. P. (2020). The importance of automated inventory systems in improving productivity. ResearchGate.
<https://www.researchgate.net>
- HashMicro. (2024). HashMicro inventory system.
<https://www.hashmicro.com/inventory>
- Hortizuela, M. R. (2019). Asset Guard: A multi-platform asset management system using QR code and geolocation. DMMMSU Research and Extension Journal, 3(1), 30-48.
<https://doi.org/10.62960/dmmmsu.v3i.18>
- inFlow Inventory. (2024). inFlow inventory management.
<https://www.inflowinventory.com/>



Intuit QuickBooks. (2024). QuickBooks Commerce.
<https://quickbooks.intuit.com/commerce/>

Llabres, E. G. (2023). Auto parts inventory management system. *International Journal of Scientific and Management Research*, 6(9), 314–329.
<https://doi.org/10.37502/IJSMR.2023.6922>

Masih, E. A. (2022). Feasibility of using QR code for registration & evaluation of training and its ability to increase response rate – the learners’ perception. *Nurse Education Today*, 111, 105305.
<https://doi.org/10.1016/j.nedt.2022.105305>

Moh Fajar, Ryfial Azhar, Yusuf Anshori, Rahma Laila, Rinianty, Nouval Trezandy Lapatta (2025) Optimization of Inventory Management with QR Code Integration and Sequential Search Algorithm: A Case Study in a Regional Revenue Office. <https://doi.org/10.30871/jaic.v9i2.891>

McCloud, R. F., Bekalu, M. A., Vaughan, T., Maranta, L., Peck, E., & Viswanath, K. (2023). Evidence for decision-making: The importance of systematic data collection as an essential component of responsive feedback. *Global Health: Science and Practice*, 11(Supplement 2).
<https://doi.org/10.9745/ghsp-d-22-0024>

Noguerra, C. P., Jr. (2023). Design and evaluation of an innovative mobile solution: QR code-based inventory monitoring system. *International Journal of Advanced Engineering and Management Research*, 8(3), 231–242. <https://doi.org/10.51505/ijaemr.2023.8317>



Odoo. (2024). Odoo inventory. <https://www.odoo.com/app/inventory>

Pabillaran, R., Enecuela, J., Sario, M., Comar, F., & Ibañez, E. (2023). The usability of QR-code electronic system in Lapu-Lapu City College: An assessment study. *Excellencia: International Multi-Disciplinary Journal of Education*, 1(3), 140-158. <https://doi.org/10.5281/zenodo.7568123>

Peddlr. (2024). Peddlr inventory for SMEs. <https://www.peddlr.io/>

QNE Software Philippines. (2024). QNE inventory management system. <https://www.qne.com.ph/inventory-software/>

Santos, M. A., & Reyes, J. D. (2021). Traditional practices in inventory management and their operational impacts.

Shirley, D. R. A., R.B., A., A, D., & M.P., K. (2021b). QR-based inventory management system (QR-IMS) of passenger luggage using website. 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), 1180–1185. <https://doi.org/10.1109/iciccs51141.2021.9432384>

SiteGiant Philippines. (2024). SiteGiant Philippines. <https://sitegiant.ph/>

StoreHub. (2024). StoreHub POS and inventory. <https://www.storehub.com/ph/>

Tan, K. H., & Lee, M. Y. (2020). QR codes in modern inventory tracking. Lightspeed HQ.



<https://www.lightspeedhq.com/blog/qr-codes-for-inventory-management/>

Yaqin, M. A., Ariyanto, S. A., Amtini, L. S., & Zhalianti, F. N. (2024). Development of a QR code-based inventory system and consumable material management using Django python. *Journal of Intelligent Computing & Health Informatics*, 5(2), 64. <https://doi.org/10.26714/jichi.v5i2.15578>

Zoho Corporation. (n.d.). Zoho inventory. <https://www.zoho.com/inventory/>

<end of page>



Appendices

APPENDIX A

EVALUATION FORM: SmartStock User Acceptance Testing (UAT)

Evaluator: _____ Role: ☐ Admin ☐ Staff Date: _____

Instructions: Rate the system from 5 (Excellent) to 1 (Poor).

CRITERIA	5	4	3	2	1
Functionality: Does the system record sales & stock accurately?	☐	☐	☐	☐	☐
QR Scanning: Is the QR code scanning fast and accurate?	☐	☐	☐	☐	☐
Usability: Is the system easy to learn and navigate?	☐	☐	☐	☐	☐
Performance: Does the system run smoothly without crashing?	☐	☐	☐	☐	☐



Security: Are restricted features (like deleting items) safe?

[]

[]

[]

[]

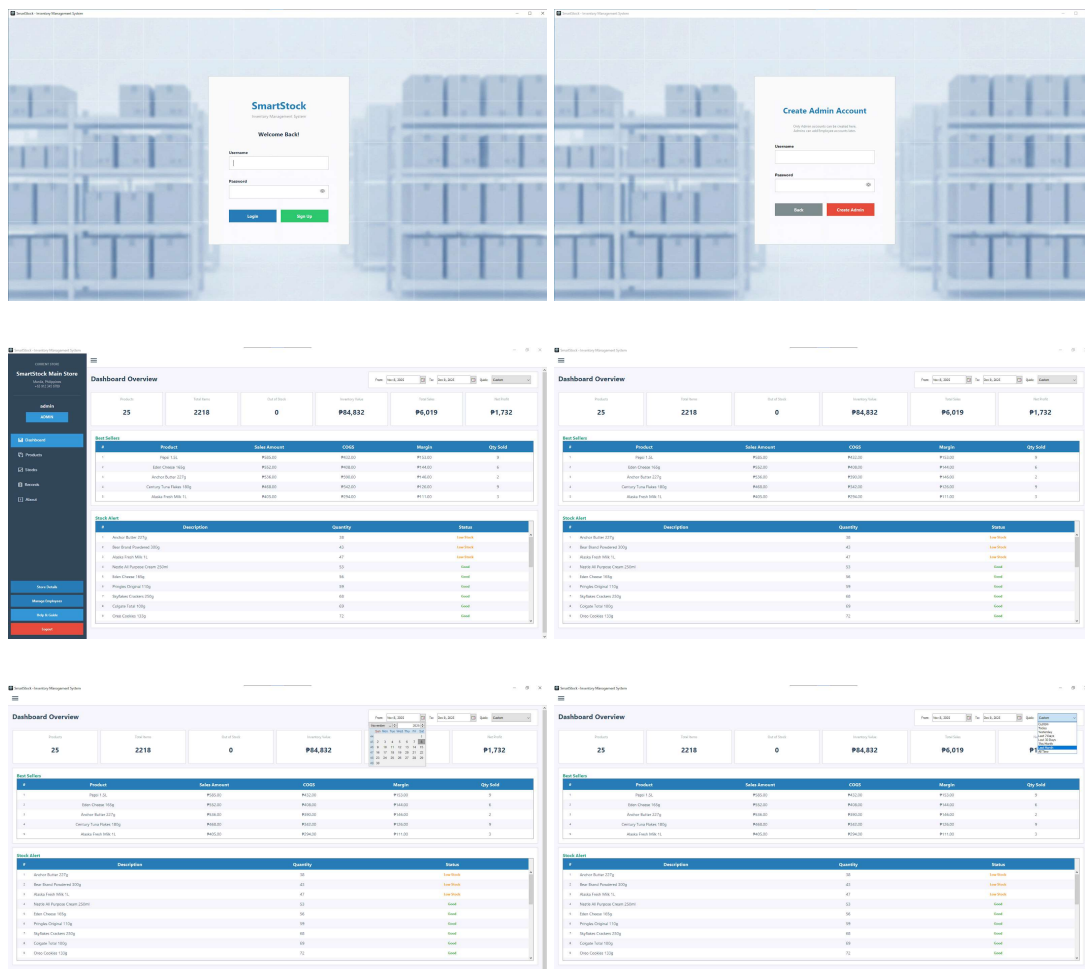
[]

Comments: _____

Signature: _____

APPENDIX B

User Interface



MANAGE PRODUCTS										Add New Manage	
#	ID	Product Name	Category	Units	Cost Price	Total Cost	Inventory	Reorder Price	Total Stock	Back	LEARNER & CONTROLS View Details
1	1	Apple 15G	Mobiles	per unit	\$1000	\$10000	10000	\$1000	100000	100	
2	2	Apple 16G	Mobiles	per unit	\$1100	\$11000	10000	\$1100	100000	100	
3	3	Apple 17G	Mobiles	per unit	\$1200	\$12000	10000	\$1200	100000	100	
4	4	Apple 18G	Mobiles	per unit	\$1300	\$13000	10000	\$1300	100000	100	
5	5	Apple 19G	Mobiles	per unit	\$1400	\$14000	10000	\$1400	100000	100	
6	6	Apple 20G	Mobiles	per unit	\$1500	\$15000	10000	\$1500	100000	100	
7	7	Apple 21G	Mobiles	per unit	\$1600	\$16000	10000	\$1600	100000	100	
8	8	Apple 22G	Mobiles	per unit	\$1700	\$17000	10000	\$1700	100000	100	
9	9	Apple 23G	Mobiles	per unit	\$1800	\$18000	10000	\$1800	100000	100	
10	10	Apple 24G	Mobiles	per unit	\$1900	\$19000	10000	\$1900	100000	100	
11	11	Apple 25G	Mobiles	per unit	\$2000	\$20000	10000	\$2000	100000	100	LEARNER & CONTROLS View Details
12	12	Apple 26G	Mobiles	per unit	\$2100	\$21000	10000	\$2100	100000	100	
13	13	Apple 27G	Mobiles	per unit	\$2200	\$22000	10000	\$2200	100000	100	
14	14	Apple 28G	Mobiles	per unit	\$2300	\$23000	10000	\$2300	100000	100	
15	15	Apple 29G	Mobiles	per unit	\$2400	\$24000	10000	\$2400	100000	100	
16	16	Apple 30G	Mobiles	per unit	\$2500	\$25000	10000	\$2500	100000	100	
17	17	Apple 31G	Mobiles	per unit	\$2600	\$26000	10000	\$2600	100000	100	
18	18	Apple 32G	Mobiles	per unit	\$2700	\$27000	10000	\$2700	100000	100	
19	19	Apple 33G	Mobiles	per unit	\$2800	\$28000	10000	\$2800	100000	100	
20	20	Apple 34G	Mobiles	per unit	\$2900	\$29000	10000	\$2900	100000	100	
21	21	Apple 35G	Mobiles	per unit	\$3000	\$30000	10000	\$3000	100000	100	LEARNER & CONTROLS View Details
22	22	Apple 36G	Mobiles	per unit	\$3100	\$31000	10000	\$3100	100000	100	
23	23	Apple 37G	Mobiles	per unit	\$3200	\$32000	10000	\$3200	100000	100	
24	24	Apple 38G	Mobiles	per unit	\$3300	\$33000	10000	\$3300	100000	100	
25	25	Apple 39G	Mobiles	per unit	\$3400	\$34000	10000	\$3400	100000	100	
26	26	Apple 40G	Mobiles	per unit	\$3500	\$35000	10000	\$3500	100000	100	
27	27	Apple 41G	Mobiles	per unit	\$3600	\$36000	10000	\$3600	100000	100	
28	28	Apple 42G	Mobiles	per unit	\$3700	\$37000	10000	\$3700	100000	100	
29	29	Apple 43G	Mobiles	per unit	\$3800	\$38000	10000	\$3800	100000	100	
30	30	Apple 44G	Mobiles	per unit	\$3900	\$39000	10000	\$3900	100000	100	
31	31	Apple 45G	Mobiles	per unit	\$4000	\$40000	10000	\$4000	100000	100	LEARNER & CONTROLS View Details
32	32	Apple 46G	Mobiles	per unit	\$4100	\$41000	10000	\$4100	100000	100	
33	33	Apple 47G	Mobiles	per unit	\$4200	\$42000	10000	\$4200	100000	100	
34	34	Apple 48G	Mobiles	per unit	\$4300	\$43000	10000	\$4300	10		



MANAGE PRODUCTS

ID	Product Name	Category	Unit	Cost Price	Total Cost	Markup %	Retail Price	Total Retail	Stock
1	Apple 12	Smartphone	unit	¥12,000	¥12,000	40%	¥16,800	¥16,800	100
2	Apple 13	Smartphone	unit	¥13,000	¥13,000	40%	¥18,200	¥18,200	100
3	Apple Watch Series 7	Wearable	piece	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
4	Apple AirPods Pro (2nd Gen)	Audio	pair	¥2,000	¥2,000	20%	¥2,400	¥2,400	100
5	Apple iPad Air (5th Gen)	Tablet	unit	¥5,000	¥5,000	40%	¥7,000	¥7,000	100
6	Apple iPad Pro (11-inch, 64GB)	Tablet	unit	¥6,000	¥6,000	40%	¥8,400	¥8,400	100
7	Apple iPad Pro (12.9-inch, 64GB)	Tablet	unit	¥8,000	¥8,000	40%	¥11,200	¥11,200	100
8	Apple TV 4K (3rd Gen, 64GB)	Streaming Device	unit	¥1,000	¥1,000	20%	¥1,200	¥1,200	100
9	Apple HomePod mini	Smart Speaker	unit	¥1,000	¥1,000	20%	¥1,200	¥1,200	100
10	Apple Watch Series 7 (41mm)	Wearable	unit	¥3,000	¥3,000	20%	¥3,600	¥3,600	100
11	Apple Watch Series 7 (45mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
12	Apple Watch Series 7 (GPS, 41mm)	Wearable	unit	¥3,000	¥3,000	20%	¥3,600	¥3,600	100
13	Apple Watch Series 7 (GPS, 45mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
14	Apple Watch Series 7 (Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
15	Apple Watch Series 7 (Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
16	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
17	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
18	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
19	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
20	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
21	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
22	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
23	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
24	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
25	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100

Product Sold

Product Name: Apple 12

Unit Price: ¥12,000

Total Price: ¥12,000

Quantity: 1

Subtotal: ¥12,000

Tax: ¥0.00

Total: ¥12,000

Payment Method: Cash

Payment Amount: ¥12,000

Change: ¥0.00

Receipt Number: 123456789

Barcode

MANAGE PRODUCTS

ID	Product Name	Category	Unit	Cost Price	Total Cost	Markup %	Retail Price	Total Retail	Stock
1	Apple 12	Smartphone	unit	¥12,000	¥12,000	40%	¥16,800	¥16,800	100
2	Apple 13	Smartphone	unit	¥13,000	¥13,000	40%	¥18,200	¥18,200	100
3	Apple Watch Series 7	Wearable	piece	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
4	Apple AirPods Pro (2nd Gen)	Audio	pair	¥2,000	¥2,000	20%	¥2,400	¥2,400	100
5	Apple iPad Air (5th Gen)	Tablet	unit	¥5,000	¥5,000	40%	¥7,000	¥7,000	100
6	Apple iPad Pro (11-inch, 64GB)	Tablet	unit	¥6,000	¥6,000	40%	¥8,400	¥8,400	100
7	Apple iPad Pro (12.9-inch, 64GB)	Tablet	unit	¥8,000	¥8,000	40%	¥11,200	¥11,200	100
8	Apple TV 4K (3rd Gen, 64GB)	Streaming Device	unit	¥1,000	¥1,000	20%	¥1,200	¥1,200	100
9	Apple HomePod mini	Smart Speaker	unit	¥1,000	¥1,000	20%	¥1,200	¥1,200	100
10	Apple Watch Series 7 (41mm)	Wearable	unit	¥3,000	¥3,000	20%	¥3,600	¥3,600	100
11	Apple Watch Series 7 (45mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
12	Apple Watch Series 7 (GPS, 41mm)	Wearable	unit	¥3,000	¥3,000	20%	¥3,600	¥3,600	100
13	Apple Watch Series 7 (GPS, 45mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
14	Apple Watch Series 7 (Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
15	Apple Watch Series 7 (Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
16	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
17	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
18	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
19	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
20	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
21	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
22	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
23	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100
24	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	¥3,500	¥3,500	20%	¥4,200	¥4,200	100
25	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	¥4,000	¥4,000	20%	¥4,800	¥4,800	100

Stock Summary

ID	Product Name	Category	Unit	Stock In	Stock Out	Balance	Status
1	Apple 12	Smartphone	unit	100	0	100	Available
2	Apple 13	Smartphone	unit	100	0	100	Available
3	Apple Watch Series 7	Wearable	piece	100	0	100	Available
4	Apple AirPods Pro (2nd Gen)	Audio	pair	100	0	100	Available
5	Apple iPad Air (5th Gen)	Tablet	unit	100	0	100	Available
6	Apple iPad Pro (11-inch, 64GB)	Tablet	unit	100	0	100	Available
7	Apple iPad Pro (12.9-inch, 64GB)	Tablet	unit	100	0	100	Available
8	Apple TV 4K (3rd Gen, 64GB)	Streaming Device	unit	100	0	100	Available
9	Apple HomePod mini	Smart Speaker	unit	100	0	100	Available
10	Apple Watch Series 7 (41mm)	Wearable	unit	100	0	100	Available
11	Apple Watch Series 7 (45mm)	Wearable	unit	100	0	100	Available
12	Apple Watch Series 7 (GPS, 41mm)	Wearable	unit	100	0	100	Available
13	Apple Watch Series 7 (GPS, 45mm)	Wearable	unit	100	0	100	Available
14	Apple Watch Series 7 (Cellular, 41mm)	Wearable	unit	100	0	100	Available
15	Apple Watch Series 7 (Cellular, 45mm)	Wearable	unit	100	0	100	Available
16	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	100	0	100	Available
17	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	100	0	100	Available
18	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	100	0	100	Available
19	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	100	0	100	Available
20	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	100	0	100	Available
21	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	100	0	100	Available
22	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	100	0	100	Available
23	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	100	0	100	Available
24	Apple Watch Series 7 (GPS + Cellular, 41mm)	Wearable	unit	100	0	100	Available
25	Apple Watch Series 7 (GPS + Cellular, 45mm)	Wearable	unit	100	0	100	Available

TRANSACTION HISTORY

ID	Date	Product Name	Unit	Type	Qty	Cost Price	Total Cost	Markup %	Total Retail
1	2024-01-01 10:00:00	Apple 12	unit	SALE	1	¥12,000	¥12,000	40%	¥16,800
2	2024-01-01 10:00:00	Apple 13	unit	SALE	1	¥13,000	¥13,000	40%	¥18,200
3	2024-01-01 10:00:00	Apple Watch Series 7	piece	SALE	1	¥4,000	¥4,000	20%	¥4,800
4	2024-01-01 10:00:00	Apple AirPods Pro (2nd Gen)	pair	SALE	1	¥2,000	¥2,000	20%	¥2,400
5	2024-01-01 10:00:00	Apple iPad Air (5th Gen)	unit	SALE	1	¥5,000	¥5,000	40%	¥7,000
6	2024-01-01 10:00:00	Apple iPad Pro (11-inch, 64GB)	unit	SALE	1	¥6,000	¥6,000	40%	¥8,400
7	2024-01-01 10:00:00	Apple iPad Pro (12.9-inch, 64GB)	unit	SALE	1	¥8,000	¥8,000	40%	¥11,200
8	2024-01-01 10:00:00	Apple TV 4K (3rd Gen, 64GB)	unit	SALE	1	¥1,000	¥1,000	20%	¥1,200
9	2024-01-01 10:00:00	Apple HomePod mini	unit	SALE	1	¥1,000	¥1,000	20%	¥1,200
10	2024-01-01 10:00:00	Apple Watch Series 7 (41mm)	unit	SALE	1	¥3,000	¥3,000	20%	¥3,600
11	2024-01-01 10:00:00	Apple Watch Series 7 (45mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
12	2024-01-01 10:00:00	Apple Watch Series 7 (GPS, 41mm)	unit	SALE	1	¥3,000	¥3,000	20%	¥3,600
13	2024-01-01 10:00:00	Apple Watch Series 7 (GPS, 45mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
14	2024-01-01 10:00:00	Apple Watch Series 7 (Cellular, 41mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
15	2024-01-01 10:00:00	Apple Watch Series 7 (Cellular, 45mm)	unit	SALE	1	¥4,000	¥4,000	20%	¥4,800
16	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 41mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
17	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 45mm)	unit	SALE	1	¥4,000	¥4,000	20%	¥4,800
18	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 41mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
19	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 45mm)	unit	SALE	1	¥4,000	¥4,000	20%	¥4,800
20	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 41mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
21	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 45mm)	unit	SALE	1	¥4,000	¥4,000	20%	¥4,800
22	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 41mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
23	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 45mm)	unit	SALE	1	¥4,000	¥4,000	20%	¥4,800
24	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 41mm)	unit	SALE	1	¥3,500	¥3,500	20%	¥4,200
25	2024-01-01 10:00:00	Apple Watch Series 7 (GPS + Cellular, 45mm)	unit	SALE	1	¥4,000	¥4,000	20%	¥4,800

Settings

Store Information

Store Name: SmartLock Main Store

Location: 123 Main Street, Quezon City

Contact Number: +639123456789

Pricing Settings

Default Markup Percentage: 20%

Save

Settings

Change Username

Current Username: admin

New Username:

Current Password:

Save Changes





APPENDIX D

Documentation



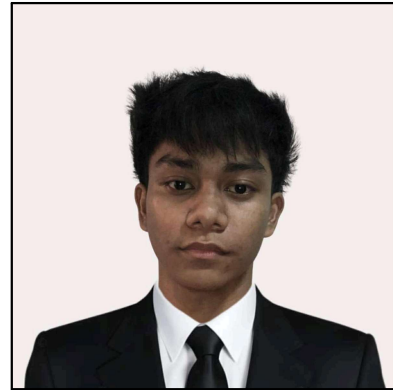


APPENDIX E

CURRICULUM VITAE

ALCANTARA, GEORGE HAROLD A.

Blk 4 Lot 2 Saint Claire Street, Santiago Subdivision,
Brgy Sta. Monica, Novaliches, Quezon City
alcantara.georgeharold.ayson@gmail.com
0946-043-2726



I. Personal Information

Date of Birth : April 8, 2005
Place of Birth : Quezon City
Citizenship : Filipino
Sex : Male
Age : 20
Civil Status : Single
Religion : Born Again

II. Education Attainment

Tertiary : Quezon City University
2025 - 2029
Secondary : Doña Rosario High School
2017 - 2023
Primary : Rosa L. Susano Elementary School
2011 - 2016



ANGELES, RON PAULO G.

Blk 11 Lot 5 Sitio Looban Brgy.Kaligayahan
Novaliches Quezon City
angeles.ronpaulo.gutierrez@gmail.com
0951-299-8322



I. Personal Information

Date of Birth : February 14, 2007
Place of Birth : Babo Sacan Porac,
Pampanga
Citizenship : Filipino
Sex : Male
Age : 18
Civil Status : Single
Religion : Roman Catholic

II. Education Attainment

Tertiary : Quezon City University
2025 - 2029
Secondary : Doña Rosario High School
2017 - 2023
Primary : Rosa L. Susano Elementary School
2011 - 2016



CALANGIAN, MATTHEW DANE D.

Jordan Park Homes, Commonwealth, Quezon City
calangianmatthewdane@gmail.com
0976-297-3244



I. Personal Information

Date of Birth : July 11, 2007
Place of Birth : Quezon City
Citizenship : Filipino
Sex : Male
Age : 18
Civil Status : Single
Religion : Catholic

I. Education Attainment

Tertiary : Quezon City University
2025 - 2029
Secondary : Lagro High School
2017 - 2024
Primary : Fairview Elementary School
2011 - 2016



JARIEL, ALDRIN MIGUEL A.

12-D Legislative St., Batasan Hills,
Quezon City
jariel.aldrin.alipao@gmail.com
0995-595-4094



I. Personal Information

Date of Birth : October 15, 2007
Place of Birth : Quezon City
Citizenship : Filipino
Sex : Male
Age : 18
Civil Status : Single
Religion : Roman Catholic

II. Education Attainment

Tertiary : Quezon City University
2025 - 2029
Secondary : Batasan Hills National High School
2017 - 2024 Mary The Queen College of Quezon City
Primary : San Diego Elementary School
2011 - 2016



PEREZ, JOHN CRISTHOPER A.

Phase 3 Block 7 Lupang Pangako
Payatas B Quezon city
perezalarcon101@gmail.com
0995-595-4094



I. Personal Information

Date of Birth : January 07, 2007
Place of Birth : Fabella Hospital
Citizenship : Filipino
Sex : Male
Age : 18
Civil Status : Single
Religion : Roman Catholic

II. Education Attainment

Tertiary : Quezon City University
2025 - 2029
Secondary : Justice Cecilia Muños Palma High School
2017 - 2024
Primary : Lupang Pangako Elementary School
2011 - 2016



APPENDIX C

Source Code

App.java

```
import com.inventorysystem.gui.userFrame;
```

```
import com.inventorysystem.gui.Startup;
```

```
import javax.swing.JFrame;
```

```
import javax.swing.SwingUtilities;
```

```
import javax.swing.UIManager;
```

```
// Main app
```

```
public class App {
```

```
    public static void main(String[] args) {
```

```
        // Splash screen
```

```
        Startup splash = new Startup();
```

```
        splash.showSplash();
```



```
SwingUtilities.invokeLater() -> {  
  
    try {  
  
        splash.updateProgress(20, "Loading UI components...");  
  
        Thread.sleep(250);  
  
        // OS look  
  
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());  
  
  
        splash.updateProgress(50, "Initializing system...");  
  
        Thread.sleep(250);  
  
        // Smooth UI  
  
        System.setProperty("awt.useSystemAAFontSettings", "on");  
  
        System.setProperty("swing.aatext", "true");  
  
  
        splash.updateProgress(80, "Starting application...");  
  
        Thread.sleep(250);  
    }  
}
```



```
} catch (Exception e) {  
  
    com.inventorysystem.util.DebugLogger.error("Failed to set LookAndFeel or  
rendering properties", e);  
  
}  
  
splash.updateProgress(100, "Ready!");  
  
try {  
  
    Thread.sleep(200);  
  
} catch (InterruptedException e) {  
  
    Thread.currentThread().interrupt();  
  
    com.inventorysystem.util.DebugLogger.error("Splash screen delay  
interrupted", e);  
  
}  
  
// Show main window  
  
splash.closeSplash();  
  
userFrame frame = new userFrame();  
  
frame.setExtendedState(JFrame.MAXIMIZED_BOTH); // Full screen  
  
frame.setVisible(true);
```



```
});  
  
}  
  
}  
  
Config.properties  
  
# Database configuration for Capstone Inventory Management System  
  
db.url=jdbc:mysql://localhost:3306/smartstock_db?useSSL=false&serverTimezone=Asia/Manila  
  
db.user=root  
  
db.password=  
  
DatabaseConnection.java  
  
package com.inventorysystem.data;  
  
  
import java.io.IOException;  
  
import java.io.InputStream;  
  
import java.sql.Connection;  
  
import java.sql.DriverManager;  
  
import java.sql.SQLException;
```



```
import java.util.Properties;

// DB connect

public class DatabaseConnection {

    private static final String DB_URL;

    private static final String DB_USER;

    private static final String DB_PASSWORD;

    // Load DB config

    static {

        Properties properties = new Properties();

        try (InputStream input =
DatabaseConnection.class.getResourceAsStream("/config.properties")) {

            if (input == null) {

                throw new RuntimeException("Cannot find config.properties file");

            }

            properties.load(input);

            DB_URL = properties.getProperty("db.url");
```



```
DB_USER = properties.getProperty("db.user");

DB_PASSWORD = properties.getProperty("db.password");

if (DB_URL == null || DB_USER == null || DB_PASSWORD == null) {

    throw new RuntimeException("Database config is incomplete in
config.properties");

}

} catch (IOException ex) {

    throw new RuntimeException("Failed to load database configuration", ex);

}

}

private DatabaseConnection() {}

// Get DB connection

public static Connection getConnection() throws SQLException {

    return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

}

}
```



UserFrame.java

```
package com.inventorysystem.gui;
```

```
import com.inventorysystem.data.StoreRepository;
```

```
import com.inventorysystem.data.UserRepository;
```

```
import com.inventorysystem.model.Store;
```

```
import javax.imageio.ImageIO;
```

```
import javax.swing.*;
```

```
import javax.swing.border.EmptyBorder;
```

```
import javax.swing.border.LineBorder;
```

```
import java.awt.*;
```

```
import java.io.File;
```

```
import java.io.IOException;
```

```
import java.sql.SQLException;
```

```
// User frame
```

```
public class userFrame extends JFrame {
```




```
private CardLayout cardLayout = new CardLayout();

private JPanel mainContainer;


public int loggedInUserId;

public String loggedInUsername;

public String loggedInUserRole;


private StoreRepository storeRepository;

private UserRepository userRepository;


public userFrame() {

    this.storeRepository = new StoreRepository();

    this.userRepository = new UserRepository();


    setTitle("SmartStock - Inventory Management System");

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    setResizable(true);


    // Set icon
```



```
try {

    Image icon = ImageIO.read(new File("src/resources/SmartStock.png"));

    // Scale icon

    java.util.List<Image> icons = new java.util.ArrayList<>();

    icons.add(icon.getScaledInstance(16, 16, Image.SCALE_SMOOTH));

    icons.add(icon.getScaledInstance(32, 32, Image.SCALE_SMOOTH));

    icons.add(icon.getScaledInstance(64, 64, Image.SCALE_SMOOTH));

    icons.add(icon.getScaledInstance(128, 128, Image.SCALE_SMOOTH));

    setIconImages(icons);

} catch (IOException e) {

    com.inventorysystem.util.DebugLogger.debug("App icon not found, using
default");

}

// For 1366x768

setSize(1366, 768);

setPreferredSize(new Dimension(1366, 768));

setMinimumSize(new Dimension(1200, 700));

setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximized
```



```
mainContainer = new JPanel(cardLayout);

mainContainer.setBackground(UIConstants.BACKGROUND_COLOR);


LoginPanel loginPanel = new LoginPanel(this);

SignupPanel signupPanel = new SignupPanel(this);


mainContainer.add(loginPanel, "login");

mainContainer.add(signupPanel, "signup");


add(mainContainer);


// Center for login

setLocationRelativeTo(null);

}


// Login method

public void handleLoginSuccess(int userId, String username, String role) {

    this.loggedInUserId = userId;
```



```
this.loggedInUsername = username;

this.loggedInUserRole = role;

try {

    Store storeToUse;

    if (role.equalsIgnoreCase("Employee")) {

        // Get admin store

        int adminId = userRepository.getAdminIdForEmployee(userId);

        if (adminId == -1) {

            JOptionPane.showMessageDialog(this,

                "Error: Your account is not linked to an Admin.\nPlease contact system administrator.",

                "Account Error", JOptionPane.ERROR_MESSAGE);

            showLoginPanel();

            return;

        }

        storeToUse = storeRepository.getStoreByUserId(adminId);

        if (storeToUse == null) {
```



```
JOptionPane.showMessageDialog(this,  
  
    "Error: Your Admin has not set up a store yet.\nPlease contact your  
administrator.",  
  
    "Store Not Found", JOptionPane.ERROR_MESSAGE);  
  
showLoginPanel();  
  
return;  
  
}  
  
} else {  
  
    // Get/create store  
  
    storeToUse = storeRepository.getStoreByUserId(userId);  
  
    if (storeToUse == null) {  
  
        storeToUse = promptAndCreateStore(userId);  
  
        if (storeToUse == null) {  
  
            showLoginPanel();  
  
            return;  
  
        }  
  
    }  
  
}
```



```
        showMainApplication(storeToUse.name(), storeToUse.location(),
storeToUse.contact());

    } catch (SQLException e) {

        JOptionPane.showMessageDialog(this, "Error fetching store: " + e.getMessage(),

            "Database Error", JOptionPane.ERROR_MESSAGE);

        showLoginPanel();

    }

}

// Main method

public void showMainApplication(String storeName, String storeLocation, String
storeContact) {

    MainApplicationPanel mainAppPanel = new MainApplicationPanel(

        this, loggedInUsername, loggedInUserRole, storeName, storeLocation,
storeContact

    );

    mainContainer.add(mainAppPanel, "mainApp");

    cardLayout.show(mainContainer, "mainApp");
```



```
// Full screen

setExtendedState(JFrame.MAXIMIZED_BOTH);

}

// UI method

private Store promptAndCreateStore(int userId) throws SQLException {

    JTextField nameField = new JTextField(20);

    nameField.setFont(UIConstants.INPUT_FONT);

    nameField.setBorder(BorderFactory.createCompoundBorder(

        new LineBorder(UIConstants.BORDER_COLOR, 1, false), // Border

        new EmptyBorder(10, 12, 10, 12)

    ));

    JTextField locField = new JTextField(20);

    locField.setFont(UIConstants.INPUT_FONT);

    locField.setBorder(BorderFactory.createCompoundBorder(

        new LineBorder(UIConstants.BORDER_COLOR, 1, false), // Border

        new EmptyBorder(10, 12, 10, 12)

    ));
```



```

JTextField contactField = new JTextField(20);

contactField.setFont(UIConstants.INPUT_FONT);

contactField.setBorder(BorderFactory.createCompoundBorder(

    new LineBorder(UIConstants.BORDER_COLOR, 1, false), // Border

    new EmptyBorder(10, 12, 10, 12)

));

JPanel panel = new JPanel(new GridBagLayout());

panel.setBackground(UIConstants.FORM_COLOR);

panel.setBorder(new EmptyBorder(20, 20, 20, 20));

GridBagConstraints gbc = new GridBagConstraints();

gbc.fill = GridBagConstraints.HORIZONTAL;

gbc.insets = new Insets(10, 10, 10, 10);

gbc.gridx = 0;

gbc.anchor = GridBagConstraints.WEST;

// Store name
```




```
gbc.gridy = 0;

JLabel nameLabel = new JLabel("Store Name: *");

nameLabel.setFont(UIConstants.LABEL_BOLD_FONT);

nameLabel.setForeground(UIConstants.TEXT_PRIMARY);

panel.add(nameLabel, gbc);


gbc.gridy = 1;

panel.add(nameField, gbc);


// Location

gbc.gridy = 2;

JLabel locLabel = new JLabel("Location: (Optional)");

locLabel.setFont(UIConstants.LABEL_FONT);

locLabel.setForeground(UIConstants.TEXT_SECONDARY);

panel.add(locLabel, gbc);


gbc.gridy = 3;

panel.add(locField, gbc);
```



```
// Contact

gbc.gridy = 4;

JLabel contactLabel = new JLabel("Contact: (Optional)");

contactLabel.setFont(UIConstants.LABEL_FONT);

contactLabel.setForeground(UIConstants.TEXT_SECONDARY);

panel.add(contactLabel, gbc);


gbc.gridy = 5;

panel.add(contactField, gbc);


while (true) {

    int result = JOptionPane.showConfirmDialog(this, panel, "Set Up Your Store",

        JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);

    if (result == JOptionPane.OK_OPTION) {

        String newName = nameField.getText().trim();

        String newLocation = locField.getText().trim();

        String newContact = contactField.getText().trim();
```



```
if (newName.isEmpty()) {

    JOptionPane.showMessageDialog(this, "Store Name cannot be empty.",

        "Input Error", JOptionPane.ERROR_MESSAGE);

    continue;

}

boolean success = storeRepository.addStore(userId, newName, newLocation,
newContact);

if (success) {

    Store newStore = storeRepository.getStoreByUserId(userId);

    if (newStore != null) {

        return newStore;

    } else {

        throw new SQLException("Failed to retrieve store after creation.");

    }

} else {

    JOptionPane.showMessageDialog(this, "Failed to add store. Please try
again.",
```



```
"Error", JOptionPane.ERROR_MESSAGE);

    }

    } else {

        // Cancelled

        JOptionPane.showMessageDialog(this,

            "Store setup is required to use the system.",

            "Setup Required", JOptionPane.INFORMATION_MESSAGE);

        return null;

    }

}

}

}

// Cancel method

public void showLoginPanel() {

    for (Component comp : mainContainer.getComponents()) {

        if (comp instanceof MainApplicationPanel) {

            mainContainer.remove(comp);

        }

    }

}
```



```
this.loggedInUserId = 0;

this.loggedInUsername = null;

this.loggedInUserRole = null;


cardLayout.show(mainContainer, "login");

setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximized

setLocationRelativeTo(null);

}


// End method


public void showSignupPanel() {

    cardLayout.show(mainContainer, "signup");

}

}


MainApplicationPanel.java

package com.inventorysystem.gui;
```



```
import com.inventorysystem.data.StoreRepository;

import javax.swing.*;

import javax.swing.border.*;

import java.awt.*;

import java.awt.event.MouseAdapter;

import java.awt.event.MouseEvent;

// Main app panel

public class MainApplicationPanel extends JPanel {

    private final JFrame mainFrame;

    private final String userRole;

    private final StoreRepository storeRepository;

    private final CardLayout cardLayout;

    private final JPanel mainContentPanel;

    private JLabel storeNameLabel;

    private JLabel storeLocationLabel;

    private JLabel storeContactLabel;
```



```
private JButton currentSelectedButton;

private JPanel navPanel; // Store reference to navigation panel

private boolean navPanelVisible = true; // Track visibility state

private JButton toggleNavButton; // Toggle button reference

public MainApplicationPanel(userFrame mainFrame, String username, String role,
                             String storeName, String storeLocation, String storeContact) {

    this.mainFrame = mainFrame;

    this.userRole = role;

    this.storeRepository = new StoreRepository();

    this.cardLayout = new CardLayout();

    setLayout(new BorderLayout());

    navPanel = createNavigationPanel(username, role, storeName, storeLocation,
storeContact);

    add(navPanel, BorderLayout.WEST);
```



```
mainContentPanel = new JPanel(cardLayout);

mainContentPanel.setBackground(UIConstants.BACKGROUND_COLOR);


mainContentPanel.add(new dashboardPanel(mainFrame), "Dashboard");

mainContentPanel.add(new productsPanel(mainFrame), "Products");

mainContentPanel.add(new stockPanel(mainFrame), "Stocks");

mainContentPanel.add(new recordsPanel(mainFrame), "Records");

mainContentPanel.add(new AboutPanel(mainFrame), "About");


JScrollPane contentScrollPane = new JScrollPane(mainContentPanel);

contentScrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_
AS_NEEDED);

contentScrollPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLL
BAR_AS_NEEDED);

contentScrollPane.setBorder(BorderFactory.createEmptyBorder());


// Create top bar with toggle button

JPanel topBar = createTopBarWithToggle();
```




```
// Wrapper panel for top bar and content

JPanel contentWrapper = new JPanel(new BorderLayout());

contentWrapper.add(topBar, BorderLayout.NORTH);

contentWrapper.add(contentScrollPane, BorderLayout.CENTER);


add(contentWrapper, BorderLayout.CENTER);


cardLayout.show(mainContentPanel, "Dashboard");
}

/**
 * Creates the top bar with hamburger menu toggle button
 */

private JPanel createTopBarWithToggle() {

    JPanel topBar = new JPanel(new BorderLayout());

    topBar.setBackground(UIConstants.WHITE);

    topBar.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0,
UIConstants.BORDER_COLOR));
```



```
topBar.setPreferredSize(new Dimension(0, 50));

// Hamburger menu button with custom paint

toggleNavButton = new JButton("") { // Empty text to avoid emoji boxes

    @Override

    protected void paintComponent(Graphics g) {

        super.paintComponent(g);

        Graphics2D g2 = (Graphics2D) g;

        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        // Draw three horizontal lines (hamburger icon)

        g2.setColor(getForeground());

        g2.setStroke(new BasicStroke(2.5f, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));

        int centerX = getWidth() / 2;

        int centerY = getHeight() / 2;

        int lineWidth = 20;
```



```
int lineSpacing = 6;

// Top line

g2.drawLine(centerX - lineWidth/2, centerY - lineSpacing,

            centerX + lineWidth/2, centerY - lineSpacing);

// Middle line

g2.drawLine(centerX - lineWidth/2, centerY,

            centerX + lineWidth/2, centerY);

// Bottom line

g2.drawLine(centerX - lineWidth/2, centerY + lineSpacing,

            centerX + lineWidth/2, centerY + lineSpacing);

}

};

toggleNavButton.setPreferredSize(new Dimension(50, 50));

toggleNavButton.setBackground(UIConstants.WHITE);

toggleNavButton.setForeground(UIConstants.TEXT_PRIMARY);

toggleNavButton.setFocusPainted(false);

toggleNavButton.setBorderPainted(false);
```



```
toggleNavButton.setContentAreaFilled(false);
```

```
toggleNavButton.setCursor(new Cursor(Cursor.HAND_CURSOR));
```

```
toggleNavButton.setToolTipText("Toggle Navigation Panel");
```

```
toggleNavButton.setFont(new Font("Segoe UI Emoji", Font.PLAIN, 24)); //
```

Fallback if custom paint fails

```
toggleNavButton.addActionListener(e -> toggleNavigationPanel());
```

```
toggleNavButton.addMouseListener(new MouseAdapter() {
```

```
    public void mouseEntered(MouseEvent e) {
```

```
        toggleNavButton.setBackground(new Color(245, 245, 245));
```

```
        toggleNavButton.setOpaque(true);
```

```
    }
```

```
    public void mouseExited(MouseEvent e) {
```

```
        toggleNavButton.setBackground(Color.WHITE);
```

```
        toggleNavButton.setOpaque(false);
```

```
    }
```

```
});
```



```
topBar.add(toggleNavButton, BorderLayout.WEST);

return topBar;

}

/**
 * Toggles the visibility of the navigation panel
 */

private void toggleNavigationPanel() {

    navPanelVisible = !navPanelVisible;

    navPanel.setVisible(navPanelVisible);

    // Update tooltip

    if (navPanelVisible) {

        toggleNavButton.setToolTipText("Hide Navigation Panel");

    } else {

        toggleNavButton.setToolTipText("Show Navigation Panel");

    }

}
```



```
revalidate();

repaint();

}

private JPanel createNavigationPanel(String username, String role, String storeName,

                                   String storeLocation, String storeContact) {

    JPanel navPanel = new JPanel(new BorderLayout());

    navPanel.setPreferredSize(new Dimension(280, 0));

    navPanel.setBackground(UIConstants.NAV_BACKGROUND);

    navPanel.setBorder(BorderFactory.createMatteBorder(0, 0, 0, 2, new Color(0, 0, 0,

30)));

    navPanel.add(createStoreInfoSection(storeName, storeLocation, storeContact,

username, role),

               BorderLayout.NORTH);

    navPanel.add(createNavigationButtons(), BorderLayout.CENTER);

    navPanel.add(createActionButtons(), BorderLayout.SOUTH);

    return navPanel;
```



```
}
```

```
private JPanel createStoreInfoSection(String storeName, String storeLocation,
```

```
String storeContact, String username, String role) {
```

```
JPanel panel = new JPanel();
```

```
panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
```

```
panel.setBackground(UIConstants.NAV_BACKGROUND);
```

```
panel.setBorder(new EmptyBorder(30, 20, 30, 20));
```

```
JLabel titleLabel = new JLabel("CURRENT STORE");
```

```
titleLabel.setFont(UIConstants.LABEL_BOLD_FONT.deriveFont(11f));
```

```
titleLabel.setForeground(UIConstants.GREY_COLOR);
```

```
titleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
```

```
panel.add(titleLabel);
```

```
panel.add(Box.createRigidArea(new Dimension(0, 12)));
```

```
storeNameLabel = new JLabel(storeName);
```

```
storeNameLabel.setFont(UIConstants.TITLE_FONT.deriveFont(20f));
```

```
storeNameLabel.setForeground(UIConstants.WHITE);
```



```
storeNameLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

panel.add(storeNameLabel);

panel.add(Box.createRigidArea(new Dimension(0, 8)));

storeLocationLabel = new JLabel(storeLocation.isEmpty() ? "No location" :
storeLocation);

storeLocationLabel.setFont(UIConstants.LABEL_FONT_PLAIN_12);

storeLocationLabel.setForeground(UIConstants.BORDER_DARK);

storeLocationLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

panel.add(storeLocationLabel);

storeContactLabel = new JLabel(storeContact.isEmpty() ? "No contact" :
storeContact);

storeContactLabel.setFont(UIConstants.LABEL_FONT_PLAIN_12);

storeContactLabel.setForeground(UIConstants.BORDER_DARK);

storeContactLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

panel.add(storeContactLabel);

panel.add(Box.createRigidArea(new Dimension(0, 25)));
```




```
JSeparator separator = new JSeparator();

separator.setForeground(new Color(127, 140, 141, 100));

separator.setMaximumSize(new Dimension(Integer.MAX_VALUE, 1));

panel.add(separator);


panel.add(Box.createRigidArea(new Dimension(0, 25)));


JLabel userLabel = new JLabel(username);

userLabel.setFont(UIConstants.LABEL_BOLD_FONT.deriveFont(16f));

userLabel.setForeground(UIConstants.WHITE);

userLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

panel.add(userLabel);


panel.add(Box.createRigidArea(new Dimension(0, 12)));

panel.add(createRoleBadge(role));


return panel;

}
```



```
private JPanel createRoleBadge(String role) {

    JPanel badge = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 0));

    badge.setMaximumSize(new Dimension(150, 36));

    badge.setPreferredSize(new Dimension(150, 36));

    badge.setBackground(UIConstants.ACCENT_COLOR);

    badge.setBorder(new EmptyBorder(8, 20, 8, 20));

    badge.setAlignmentX(Component.CENTER_ALIGNMENT);

    JLabel roleLabel = new JLabel(role.toUpperCase());

    roleLabel.setFont(UIConstants.LABEL_BOLD_FONT.deriveFont(13f));

    roleLabel.setForeground(UIConstants.WHITE);

    badge.add(roleLabel);

    return badge;

}

// Main navigation buttons (Dashboard, Products, etc.)

private JPanel createNavigationButtons() {
```



```
JPanel panel = new JPanel();

panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));

panel.setBackground(UIConstants.NAV_BACKGROUND);

panel.setBorder(new EmptyBorder(10, 10, 10, 10));

// Create all buttons

JButton dashboardBtn = createNavButton("Dashboard", "Dashboard");

JButton productsBtn = createNavButton("Products", "Products");

JButton stocksBtn = createNavButton("Stocks", "Stocks");

JButton recordsBtn = createNavButton("Records", "Records");

JButton aboutBtn = createNavButton("About", "About");

// Add buttons with spacing

panel.add(dashboardBtn);

panel.add(Box.createRigidArea(new Dimension(0, 5)));

panel.add(productsBtn);

panel.add(Box.createRigidArea(new Dimension(0, 5)));

panel.add(stocksBtn);

panel.add(Box.createRigidArea(new Dimension(0, 5)));
```



```
panel.add(recordsBtn);

panel.add(Box.createRigidArea(new Dimension(0, 5)));

panel.add(aboutBtn);

panel.add(Box.createVerticalGlue());

// Dashboard is selected by default

currentSelectedButton = dashboardBtn;

updateButtonSelection(dashboardBtn);

return panel;
}

private JButton createNavButton(String text, String panelName) {

    String icon = getNavIcon(text);

    JButton button = createBaseButton(icon + " " + text,
    UIConstants.NAV_BACKGROUND, new Color(206, 214, 224),
    UIConstants.NAV_HOVER, false);

    button.setFont(UIConstants.FONT_EMOJI_LARGE.deriveFont(Font.BOLD,
    14f));
```



```
button.addActionListener(e -> {

    cardLayout.show(mainContentPanel, panelName);

    updateButtonSelection(button);

});

return button;

}

private String getNavIcon(String text) {

    switch(text) {

        case "Dashboard": return "📊";

        case "Products": return "📦";

        case "Stocks": return "📈";

        case "Records": return "📋";

        case "About": return "ℹ️";

        default: return "●";

    }

}
```



```
// Highlight the selected button

private void updateButtonSelection(JButton selectedButton) {

    // Reset old button

    if (currentSelectedButton != null) {

        currentSelectedButton.setForeground(new Color(206, 214, 224));

        currentSelectedButton.setBackground(UIConstants.NAV_BACKGROUND);

        currentSelectedButton.setOpaque(false);

    }

    // Highlight new button

    currentSelectedButton = selectedButton;

    selectedButton.setForeground(Color.WHITE);

    selectedButton.setBackground(UIConstants.ACCENT_COLOR);

    selectedButton.setOpaque(true);

}

// Bottom action buttons (Store Settings, Logout, etc.)

private JPanel createActionButtons() {

    JPanel panel = new JPanel();
```



```
panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));

panel.setBackground(UIConstants.NAV_BACKGROUND);

panel.setBorder(new EmptyBorder(10, 10, 15, 10));


// Only admins can see these buttons

if (userRole.equalsIgnoreCase("Admin")) {

    JButton storeDetailsBtn = createActionButton("Store Details",

        UIConstants.PRIMARY_COLOR, UIConstants.PRIMARY_DARK);

    storeDetailsBtn.addActionListener(e -> openStoreSettings());

    panel.add(storeDetailsBtn);

    panel.add(Box.createRigidArea(new Dimension(0, 8)));


    JButton manageEmployeesBtn = createActionButton("Manage Employees",

        UIConstants.PRIMARY_COLOR, UIConstants.PRIMARY_DARK);

    manageEmployeesBtn.addActionListener(e -> openEmployeeManager());

    panel.add(manageEmployeesBtn);

    panel.add(Box.createRigidArea(new Dimension(0, 8)));

}
```



```
// Help button (available to all users)

JButton helpBtn = createActionButton("Help & Guide",

    new Color(52, 152, 219), new Color(41, 128, 185));

helpBtn.addActionListener(e -> openHelpDialog());

panel.add(helpBtn);

panel.add(Box.createRigidArea(new Dimension(0, 8)));


// Everyone can logout

JButton logoutBtn = createActionButton("Logout",

    UIConstants.DANGER_COLOR, UIConstants.DANGER_DARK);

logoutBtn.addActionListener(e -> logout());

panel.add(logoutBtn);


return panel;
}


// Create action button with colors

private JButton createActionButton(String text, Color bgColor, Color hoverColor) {

    return createBaseButton(text, bgColor, Color.WHITE, hoverColor, true);
}
```




```
}

// Helper method to consolidate button creation logic

private JButton createBaseButton(String text, Color bgColor, Color fgColor, Color
hoverColor, boolean isOpaque) {

    JButton button = new JButton(text);

    button.setFont(UIConstants.BUTTON_FONT.deriveFont((bgColor ==
UIConstants.NAV_BACKGROUND) ? 14f : 12f));

    button.setForeground(fgColor);

    button.setBackground(bgColor);

    button.setFocusPainted(false);

    button.setBorderPainted(false);

    button.setContentAreaFilled(false); // Key for hover effects

    button.setOpaque(isOpaque);

    button.setHorizontalAlignment(bgColor == UIConstants.NAV_BACKGROUND ?
SwingConstants.LEFT : SwingConstants.CENTER); // Alignment differs

// Size handling

if (bgColor == UIConstants.NAV_BACKGROUND) {

    button.setMaximumSize(new Dimension(Integer.MAX_VALUE, 48));
```



```
button.setPreferredSize(new Dimension(240, 48));

button.setBorder(new EmptyBorder(12, 25, 12, 25));

} else {

    button.setMaximumSize(new Dimension(Integer.MAX_VALUE, 42));

    button.setPreferredSize(new Dimension(220, 42));

    button.setBorder(new EmptyBorder(10, 18, 10, 18));

}

button.setCursor(new Cursor(Cursor.HAND_CURSOR));

// Common MouseListener for Hover Effects

button.addMouseListener(new MouseAdapter() {

    public void mouseEntered(MouseEvent e) {

        // Logic for Nav Buttons (Selection state check)

        if (bgColor == UIConstants.NAV_BACKGROUND) {

            if (button != currentSelectedButton) {

                button.setBackground(hoverColor);

                button.setForeground(Color.WHITE);

                button.setOpaque(true);
```



```
    }

}

// Logic for Action Buttons (Simple hover)

else {

    button.setBackground(hoverColor);

    button.setOpaque(true);

}

}

public void mouseExited(MouseEvent e) {

    // Logic for Nav Buttons

    if (bgColor == UIConstants.NAV_BACKGROUND) {

        if (button != currentSelectedButton) {

            button.setForeground(fgColor);

            button.setOpaque(false);

            button.setBackground(bgColor);

        }

    }

    // Logic for Action Buttons
```



```
        else {  
  
            button.setBackground(bgColor);  
  
        }  
  
    }  
  
});  
  
    return button;  
  
}  
  
// ===== BUTTON ACTIONS =====  
  
// Open store settings dialog  
  
private void openStoreSettings() {  
  
    StoreSettingsDialog dialog = new StoreSettingsDialog(mainFrame,  
storeRepository,  
  
        storeNameLabel, storeLocationLabel, storeContactLabel);  
  
    dialog.setVisible(true);  
  
}  
  
// Open employee management dialog
```



```
private void openEmployeeManager() {  
  
    EmployeeManagerDialog dialog = new EmployeeManagerDialog(mainFrame);  
  
    dialog.setVisible(true);  
  
}  
  
// Open help and user guide dialog  
  
private void openHelpDialog() {  
  
    HelpDialog helpDialog = new HelpDialog(mainFrame);  
  
    helpDialog.setVisible(true);  
  
}  
  
// Logout confirmation  
  
private void logout() {  
  
    int choice = JOptionPane.showConfirmDialog(mainFrame,  
  
        "Are you sure you want to logout?", "Confirm Logout",  
  
        JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);  
  
    if (choice == JOptionPane.YES_OPTION) {  
  
        mainFrame.showLoginPanel();  
  
    }  
}
```



```
}
```

```
// Main method for testing
```

```
public static void main(String[] args) {
```

```
    SwingUtilities.invokeLater(() -> {
```

```
        JFrame frame = new JFrame("MainApplicationPanel Test");
```

```
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        frame.setSize(1400, 900);
```

```
        userFrame mockFrame = new userFrame();
```

```
        mockFrame.loggedInUserId = 1;
```

```
        mockFrame.loggedInUsername = "Admin";
```

```
        mockFrame.loggedInUserRole = "Admin";
```

```
        MainApplicationPanel panel = new MainApplicationPanel(
```

```
            mockFrame, "Admin", "Admin",
```

```
            "Test Store", "123 Main St", "555-1234"
```

```
        );
```



```
frame.add(panel);  
  
frame.setLocationRelativeTo(null);  
  
frame.setVisible(true);  
  
});  
  
}  
  
}
```